

A Quantum-Safe Private Group System for Signal from Key Re-Randomizable Signatures

Graeme Connell
Signal Messenger

Sebastian Faller
IBM Research Europe - Zurich, ETH Zurich

Felix Günther
IBM Research Europe - Zurich

Julia Hesse
IBM Research Europe - Zurich

Vadim Lyubashevsky
IBM Research Europe - Zurich

Rolfe Schmidt
Signal Messenger

Abstract

Instant messaging services are an integral part of today’s communication and their privacy has wide societal implications. Major messengers deploy end-to-end encryption, hiding message contents from the service provider. Group messaging, however, creates the challenge of also keeping the group membership list private.

The Signal messenger currently implements private group management using techniques inspired by Chase, Perrin, and Zaverucha (CCS 2020). Transitioning this system to quantum-safe turns out to be challenging: While one-to-one messaging can often adopt the newly standardized KEMs and signatures in a relatively direct way, private group management is more complex. Signal’s existing design heavily relies on the discrete-log structure to combine anonymous credentials, verifiable encryption, and oblivious PRFs for privacy and functionality. Quantum-safe versions of these components are unfortunately, typically far less efficient, requiring heavy zero-knowledge proofs and large communication per group operation. As a result, simply “swapping in” quantum-safe primitives is unlikely to yield an optimal protocol.

This paper reconsiders the design of the entire group system from the ground-up. Our result is a scheme that possesses the same strong privacy guarantees, but is built in a more modular way using simpler underlying cryptographic building blocks that permit a more efficient quantum-safe instantiation. The modularity of our protocol further allows for gradual migration to quantum-safe: we can immediately transition components vulnerable to harvest-now-decrypt-later attacks (such as classical public-key encryption, computationally hiding commitments, etc.) while deferring the transition of other building blocks, such as authentication. We prove our design secure in an extended security model that more comprehensively captures the rich feature set of Signal’s group messaging system.

1 Introduction

Signal is an open-source messaging service, used by tens of millions of users [BBC], that is known for putting an emphasis on end-to-end-encryption and user privacy. In addition to conventional one-to-one messaging, Signal allows users to create groups and exchange messages in these groups. In a typical implementation of group messaging, the server would centrally store important information such as a list of group members, the group’s name, or the group’s image. These data items can clearly be sensitive, and might even be equally attractive targets for adversaries as the chat messages themselves. In 2019, Signal introduced their private group system [CPZ20, Ole] that hides group data from the server while still allowing the server to perform fine-grained access control and spam protection. A private group system (PGS) allows the realization of many complex operations on the group, differentiating between admins and normal users, mediated by a central server. Beyond core operations such as admins creating groups and adding/removing members, these include allowing users to leave a group on their own and updating their user data, as well as (depending on policies) to join a group via an invite link, to add other users, or to change group attributes.

Together with the desired privacy guarantees against the server, such a private group system requires mechanisms beyond what group messaging protocols such as MLS can offer. The main security goals of Signal’s private group system are the following:

- *Privacy of group membership.* The server does not learn which users are members of which groups.
- *Consistency.* All honest users should have a consistent view of who is a member of the group (and who is not anymore).
- *Anonymous group management.* Only users with the necessary privileges can create groups, add or remove members, or change the group’s name. Yet, when the server authorizes such users, their identity

must remain hidden.

- *Confidential key material.* The system facilitates key material used to encrypt users’ profile data or authorize sending group messages, accessible only to users that share a group.
- *Authorized addition.* To decrease the risk of spam groups, group administrators can only add users they know (and that did not block them).

Signal’s current private group system. Signal’s current system is based on the work by Chase, Perrin, and Zaverucha [CPZ20]. The system uses advanced cryptographic primitives to achieve the security goals described above: the list of group members is encrypted using *deterministic verifiable encryption*, and the identity of users is proven with *key-verification anonymous credentials* (KVACs). Simplifying, one can think of the server checking a user’s privilege to, e.g., add another user as follows: Using a zero-knowledge proof (ZKP), the user proves to the server that 1) the user possesses a credential that their identity corresponds to one entry of the encrypted member list, 2) the user possesses a credential that proves that the user and the new group member were in contact before, and 3) the user provides a correct encryption that contains the newly added member’s information. Signal recently added a layer of spam protection: The Sealed Sender Authorization Pass system [Sig25] allows the server to recognize legitimate group messages, without revealing the sender’s identity (as Signal is using sealed-sender sending) and without the server learning for which group the message is intended. To this end, Signal combines an oblivious pseudo-random function (OPRF) and their verifiable encryption scheme in a non-black-box manner. Concretely, the scheme exploits that the DH-based OPRF and the ElGamal-based verifiable encryption work over the same group.

The quantum-safe challenge. In Signal’s current system, the group list and all additional information about the group, like the name or the image of the group, are encrypted with an ElGamal-like encryption scheme. This sensitive information is hence an attractive target for so-called harvest-now-decrypt-later (HNDL) attacks: An adversary that gets hold of the classically encrypted group information *today* could store this data until a cryptographically-relevant quantum computer is available and decrypt it then. The possibility of such an attack being mounted *now* mandates a timely transition to a quantum-resistant system.

Given the complexity of the current system and the non-black-box use of the employed primitives, simply replacing all the primitives by quantum-safe candidates such as [BLNS23, LNP22, AG24, BDFH25] will likely be too expensive. For the OPRF-based Sealed Sender Autho-

rization Pass, it is even unclear if we can find a quantum-safe replacement, as current OPRFs [AG24, BDFH25] do not seem to allow for the same strong coupling with the verifiable encryption. This could completely roadblock the transition. Beyond the mere instantiation issues, there is a further challenge: when transitioning their secure messaging protocols to quantum safety, Signal has set a requirement of providing *hybrid* security for any public-key primitives. Due to the use of more heavy-weight primitives, such hybridization seems out of reach for the current system.

1.1 Our Work

We reconsider the design of the group management protocol from the ground-up. The goal is to have a protocol that can be immediately secure against harvest-and-decrypt attacks using efficient quantum-safe primitives, while allowing other aspects of the scheme to, for now, remain sound only with respect to classical attacks. The rationale is that protocol parts involving, for example, authentication, which only need to be secure at the moment they are executed, can wait to be upgraded to their post-quantum versions once quantum computers capable of breaking them actually exist. The amount of data sent between the Signal servers (which run on multiple cloud providers) and clients directly affects Signal’s operational costs, and so one ideally would like to wait to upgrade to quantum-safe protocols, which are all more bandwidth-intensive, only when necessary. On the other hand, it is also important to be able to easily switch to quantum-safe without redesigning the protocol. Our modular design makes it possible to swap out components for one part of the system without affecting the others. It also allows for easy testing of the fully quantum-safe protocol to determine its viability when the time to switch finally arrives.

The main conceptual difference between Signal’s current group management structure, based on [CPZ20], and our proposal is that in the former, the server acts as a “gatekeeper” making sure that all changes to the group structure follow the prescribed policy. In our proposal, on the other hand, it is the group members themselves (or more specifically, their client software) that check that the list entries are valid. The server’s main role is simply to check that members of the group (either administrators or plain users) are the ones writing the entries in the positions of the list where they are allowed. This new approach is able to retain all the operability and privacy of the prior, with the main advantage being that most operations require only basic cryptographic primitives. In particular, we only use standard symmetric- and public-key encryption schemes that can be instantiated quantum-safely, making the scheme immediately secure

against harvest-and-decrypt attacks.

Very importantly, the system is set up such that group members are able to trace any activity to the specific group member who initiated it, and so any misbehavior is immediately detected by everyone in the group. Any honest administrator detecting invalid operations (either due to malicious actors or, more likely, buggy software) can restore the group system to a valid state. A non-administrator can identify the misbehaving party and inform them that their software is possibly buggy. Crucially, such tracing must not interfere with privacy, and we prudently use a key re-randomizable signature scheme (akin to [FKM⁺16]) to ensure group members can sign under pseudonym keys (allowing the server to perform certain authorization checks) while hiding their identity from group outsiders. Key re-randomizable signature schemes follow from Schnorr signatures or a tweak of the new ML-DSA (Dilithium) [BDK⁺21] PQC NIST standard, which we introduce.¹

We formalize and analyze the desired security of a private group system through a Universal Composability (UC) [Can01] security model. Revisiting the model of [CPZ20], we ultimately chose to re-design the security model from the ground-up, not least to capture more fine-grained security guarantees that our system can provide. These include the distinction between admins and regular users (and the effects of corrupting a regular, but non-admin user), the limited power a corrupt server should have, and the ability to refresh key material of both groups and users.

1.2 Paper Outline

In Section 2, we introduce the functionality of private group systems in general and the specifics of Signal and the overall threat model, before providing an overview of our design, its rationale, and core components. In Section 3, we then present the technical details and protocol descriptions of our system. We introduce our security model for private group systems in Section 4 and relate it to the prior work of [CPZ20]. We close with deployment considerations in Section 5, mapping our design to the advanced features of Signal’s messaging architecture and discussing challenges and trade-offs.

2 System Overview

A group system is deployed between a *central server* and arbitrarily many *users*. The *core functionality* of a

¹As a side note of independent interest, since key re-randomizable signatures immediately imply (c.f. [DFL19]) (non-hierarchical) deterministic wallets [But13] used in the bitcoin BIP32 proposal [Wik13], our construction gives a quantum-safe version of deterministic wallets with signature sizes being the same as those of ML-DSA.

Core Functionality	Extended Functionality
RegisterUser (3)	Refresh (8)
GrantAcquaintance (4)	Leave (8)
Create (5)	UserAdd (8)
Fetch (6)	ChangeSettings* (13)
AddMember* (7)	ChangeAttr ^(*) (8)
RemoveMember* (12)	ToggleJoinLink* (7.7)
SetAdminRights* (7.4)	JoinViaLink (8)
	Invite* (7.6)
	RevokeInvitation* (7.6)
	AcceptInvitation (7.6)
	RotateGroupKey (14)

Table 1: Core functionality of a private group system (left) and extended functionality (right), with numbers in parentheses referring to the figure or appendix defining them. Operations marked with * require admin privileges; those with ^(*) optionally so, depending on group settings.

group system is to maintain a consistent user list of the current members of each group, distinguishing between regular users and those with admin privileges. This list is managed through a set of *core operations* (cf. Table 1) which allow users to create a new group (Create) and fetch the current version of an existing group (Fetch), as well as admins to add and remove group members (AddMember / RemoveMember) and toggle admin privileges (SetAdminRights). As a prerequisite, users need to set up an account (RegisterUser) and establish “acquaintance” with each other (GrantAcquaintance), ensuring users can only add users they know to a group.

To support additional features and security properties, Signal’s private group system further encompasses the following *extended functionality*. Users can refresh their key material (Refresh), used in particular to block other users, and leave a group on their own (Leave). The group settings can be modified by admins (ChangeSettings), to allow regular users to add other users (via UserAdd) or modify the group’s attributes, such as name, picture, etc. (ChangeAttr). Admins can activate (or deactivate) a join link (ToggleJoinLink) which allows anyone with this link to join the group (JoinViaLink), possibly requiring approval (ApproveJoin). Finally, admins can invite users they are not acquainted with to a group (Invite) and revoke such an invitation (RevokeInvitation); the invited user can choose to accept or decline such an invitation (AcceptInvitation).

2.1 Threat Model

In a private group system, we deal with a potentially corrupted server, corrupted group members, and even corrupted group administrators. *Privacy of group mem-*

bership is required to hold against even a corrupt server, while no privacy can be guaranteed if any member of the group is corrupted and can leak the whole group state. We require *consistency* to hold in the presence of corrupt group members. A corrupt server can always withhold updates from members, but crucially cannot edit the group state or roll back to previous versions of the group. Consistency is lost in case a corrupt server colludes with a group member, reflecting the fact that a server in possession of the group key material can possibly rewrite the group list arbitrarily. *Anonymity* in group management is required to hold against a corrupt server, but only as long as no corrupt group members leak any information. Admin privileges cannot be faked by corrupt members as long as they do not collude with the server. *Confidentiality of key material* and *authorized addition* are both guaranteed in the presence of a corrupt server.

2.2 Group Data Structure

At the heart of our private group system design is an evolving data structure, stored by the server and updated by group admins and regular group members through the operations described above; see Figure 1 for an illustration of that data structure. For privacy reasons, most of this data structure (e.g., user identifiers and key material) needs to be encrypted to protect it from the server. This data structure can be conceptually split into a *core* part and an *appendix*, as well as an *invite queue*. Only group admins can modify the core part, with each change constituting a new *major version* of the group, while users can add entries to the appendix, with each addition constituting a new *minor version*.

Key re-randomizable signature scheme. While in Signal’s current protocol, anonymous credentials are centrally used to privately prove a user’s membership in a list (needing to be anonymous so that the identity is not traced between lists), our protocol achieves this task using much simpler primitives. In particular, the only public-key primitive we need is (a relaxation of) a signature scheme with re-randomizable keys [FKM⁺16]. Such schemes can be constructed from Schnorr signatures or ML-DSA / Dilithium (which we will explain in Section 6) if one wishes to achieve quantum-safety; combining both further enables *hybrid* security if desired.

A re-randomizable signature scheme consists of algorithms

1. $\text{KeyGen}() \rightarrow (s, t)$, generating random secret / public “base” values;
2. $\text{RandSK}(s, \rho) \rightarrow sk$, deterministically deriving a signing key from s with randomness ρ ;

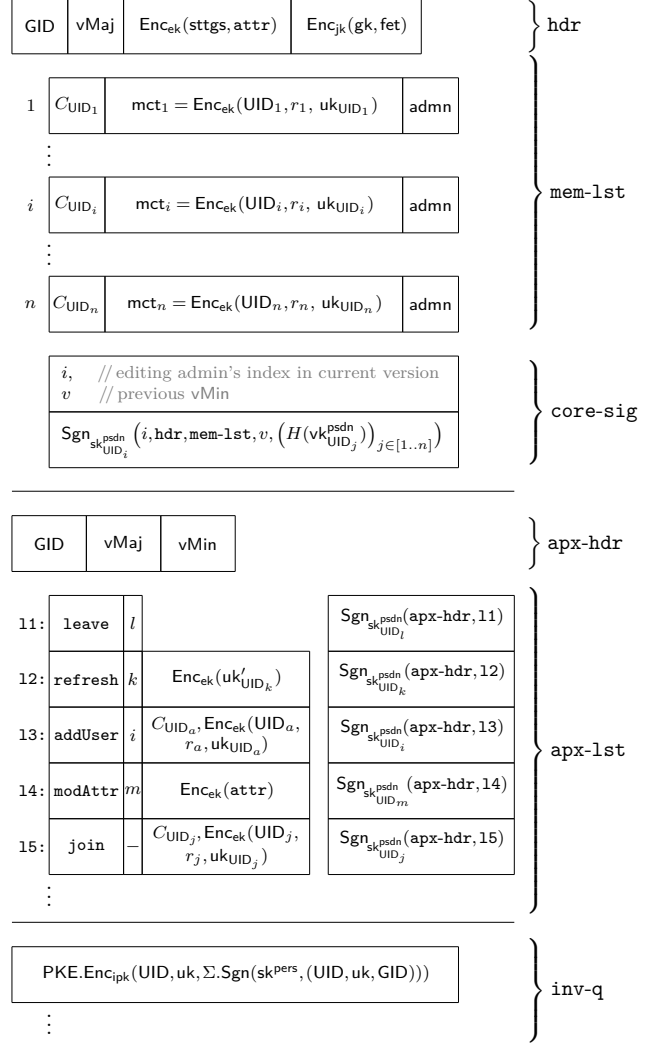


Figure 1: Illustration of the group data structure with core member data (top), appendix (middle), and invite queue (bottom). In the appendix, 11 denotes the contents of appendix line i without the signature. The server additionally stores the pseudonym verification key vk^{psdn} of each member, and a fetching token fet .

3. $\text{RandVK}(t, \rho) \rightarrow vk$, deterministically deriving a verification key from t with randomness ρ ;
4. $\text{Sgn}(sk, m) \rightarrow \sigma$, signing a message m using sk ; and
5. $\text{Vfy}(vk, m, \sigma) \rightarrow 0/1$, verification using vk .

The correctness notion that we need is that (sk, vk) derived from a base (s, t) with the same randomness is a valid key pair for a signature scheme—i.e., $\text{Vfy}(vk, m, \text{Sgn}(sk, m)) = 1$. The privacy notion is that for any two t, t' , given oracle access to RandVK and Sgn , when ρ is chosen at random each time, one cannot distinguish whether the transcript was produced when using

t or t' . The unforgeability notion states that one cannot forge signatures under any produced vk , even if one knows (and even chooses) ρ .²

In a nutshell, we will have users re-randomize their base keys for each group, using them as pseudonym signing keys for which other group members can re-derive the public key. This allows all other members in the group to link operations to the acting user, while the server merely is able to check that a signature verifies under an (unlinkable) pseudonym public key (i.e., are done by a group member).

The core. The core part of our group data structure begins with a *header*, carrying the group identifier GID , current major version, the AEAD-encrypted group settings and attributes (such as group name, group picture, etc.), and an encryption of the group key. The group key gk is refreshed with every major version, and communicated out-of-band to added members. A random fetching token fet is sampled for every major version, used as a bearer token towards the server to authorize fetching a group.

The primary component is then the *member list*, which contains a *member entry* per group member with their relevant information (notably, the member’s encrypted user identifier UID and user key uk , and a server-visible flag $admn$ marking admin members). It further contains a statistically hiding commitment $C\{UID\}$ to the user identifier, allowing optional linking to an anonymous credential to implement various policies (e.g., user key recovery or liveness checks; see Section 5 and Section 6 for further discussion and details).

With each line, we also associate the group member’s pseudonym verification key vk^{psdn} , which is unique to each group that the user belongs to, and to which only that user knows the corresponding secret key sk^{psdn} . The pseudonym key can be re-derived by members; to the server, it is given by the user adding the respective member entry. All operations that the user performs in the group will be signed with sk^{psdn} , which proves to the server that the operation is being done by the group member in the particular line, and makes this user identifiable (only) by other group members.

These keys are generated using the re-randomizable signature scheme. In particular, every Signal user with a unique user identifier UID has a static secret / public pair (sk_{base}, vk_{base}) generated using the KeyGen algorithm. Each group also has a secret re-randomization seed rrs , which is derived from the group key. The sk^{psdn} and vk^{psdn} keys are *deter-*

ministically generated as $RandSK(sk_{base}, H(UID, rrs))$ and $RandVK(vk_{base}, H(UID, rrs))$, respectively. In other words, all group members can compute each other’s pseudonym public keys. The security properties of the re-randomizable signature imply that all users are unlinkable to the server across groups and that only the user with sk^{psdn} can sign under vk^{psdn} .

The core part of the group data structure may only be edited by group admins, which is ensured by having the editing admin sign it with its pseudonym key in the *core signatures* section. This signature will be checked by the server against the claimed admin’s member entry (esp. ensuring that the member indeed has admin privileges), as well as by all members to ensure knowledge of which admin performed the update. Beyond the header and member list, the signature also covers all members’ pseudonym keys (ensuring server and users agree on the correct, unique keys), as well as the editing admin’s index and the latest previous minor version (to ensure no gaps are introduced).

Jointly, this enables the system’s core operations: allowing admins to create groups (*Create*), add members (*AddMember*), remove members (*RemoveMember*), and assign admin privileges (*SetAdminRights*), and members to fetch the current group state (*Fetch*).

The appendix. So far, only admins can modify the group list. To enable the extended functionality described above (like allowing regular members to, e.g., leave a group, refresh their user keys, or add users to a group), we introduce an *appendix* to which any regular member can (only) append entries. The appendix has an *appendix header*, carrying the group id and major and minor version. The *appendix list* then consists of entries for one of the following operations, each carrying a signature of the performing user under their pseudonym key:

- **leave:** a member leaving the group.
- **refresh:** a member refreshing their user key, accompanied by the encrypted new key.
- **addUser:** a member adding a new user, accompanied by that user’s encrypted member entry.
- **modAttr:** a member modifying the group attributes.
- **join:** a member joining via a join link, accompanied by that user’s encrypted member entry.

The appendix should be seen as capturing an intermediate state between core member list updates. That is, once an admin comes online, they will reconcile the operations in the appendix to create a new version of the core member list. The admin who modifies the core list signs the entire list with his pseudonym key, and is thus “responsible” for the correctness of the whole list. Users who append lines to the appendix individually just sign that line, limiting their responsibility (and editing capabilities).

²The difference with [FKM+16] is that we do not necessarily require (s, t) to be a signature key-pair, and we do not need the produced sk to have the same distribution as s . But we do want the signatures not to be traced to the public base t .

The intuition for splitting the group description into two parts is to prevent regular users from “destroying” the group³ by continuously rewriting the group list to the point where the server, which perhaps stores several prior versions of the group structure, no longer has any valid backups. While everyone in the group will detect that the list is invalid, and even know who made the invalid changes, their only recourse to reconstruct a valid group state is to somehow reconstruct it from their valid state. This situation, however, becomes rather messy, and so we believe that avoiding it altogether is a much cleaner and simpler solution. The appendix, being append-only, can contain invalid lines, but they are simply ignored by everyone, and the originator of these lines can be traced back.

The join queue. If join requests have to be approved, joining users enter themselves into a join queue by encrypting their user identifier and user key under a public-key encryption key ipk , and signing the request. Group members hold the corresponding decryption key, allowing them to decide on join requests and add approved users.

2.3 Core Group Operations

We describe all the group operations in Section 3, and only sketch at a very high level the main group operations for adding and removing members here in order to give a flavor of how the system works.

Adding a user to the main list. To add a user, the admin adds a new line to the core list for that user and sends it to the server together with the new user’s derived vk^{psdn} . The admin also signs the entire list under their own pseudonym key. The server checks the validity of the signature and checks that the line corresponding to the key under which it is signed is indeed an admin line allowed to perform this operation. The admin then sends the group identifier and group key material (incl. gk) to the added user over a secure one-to-one channel (in Signal, this is done within the core messaging protocol).

Note that at this point, all the server attests to is that an admin made a change and that they now know the vk^{psdn} of the newly added user—nothing about the validity of the other information in the line is known to the server because it is encrypted under the group key gk . Thus, if the ciphertext is malformed, the blame falls on the admin who signed it. When the other users fetch the new line, they check that the encrypted information contains “correct” information about the new user and that the signature is valid.

³We do not consider an admin “destroying” the group to be an attack because an admin can legally simply delete the group.

Removing a user from the main list. Removing a user requires that the group key gk be changed, and the group data structure will need to be re-encrypted under the new key. The admin thus needs to completely rewrite the group list. They will do this by simply removing that user’s line and re-encrypting all the others. The admin will also need to let the server know all the vk^{psdn} of the users in the group, which have changed since the new group key implies a new re-randomization seed rrs . The admin sends this new group description, signed under their sk^{psdn} , to the server. When the group members fetch the new version, they verify that all the lines contain “correct” information about the users and that the signature verifies. Note that verifying the signature requires recomputing the vk^{psdn} of all the users. While this operation is relatively fast (tens of milliseconds for a few hundred users using classical or ML-DSA signatures), one only needs to perform it after a member has been removed, thus it is also not very frequent.

Adding a user by a non-admin. If a non-admin group member wishes to add another group member to the group (and this operation is permitted as per the group settings), the user performs the exact same action as what an admin does above, except the user’s line is not stored in the member list, but in the appendix. This line, together with the vk^{psdn} of the new user, is then individually signed.

Leaving the group. A group member wishing to leave the group publishes a leave entry to the appendix, pointing to their member entry in the main list (or in the appendix) and signing using that entry’s pseudonym key. Messages are thus no longer sent to this user by others, and when an admin comes online, the user’s entry in the main list is removed, and the group key is rotated.

3 A New Private Group System

Let us now describe the protocols forming our new private group system in more detail.

Key hierarchies. Our protocols involve a range of user- and group-specific keys, which are derived using a key derivation function KDF in a hierarchical way, illustrated in Figure 2.

Users hold a main user key muk , from which they derive a (per-epoch) user key uk . A user key is shared with acquaintances and other members of a group, used to derive (a) an acquaintance tag acq that allows to validate another user’s key, and (b) an exporter key $expKey$ from which further application key material can be derived

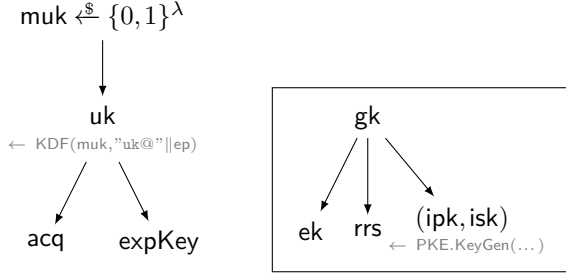


Figure 2: Key hierarchy for user keys (top) and group keys (bottom, boxed).

(e.g., a profile key for encrypting the user’s profile information, or a sending token used as endorsements for message delivery). The user key (and hence all derived keys) can be refreshed to a new epoch, allowing, for example, to reset acquaintance with (i.e., blocking) other users.

Group members share a group key gk , refreshed with every major version, which is used to derive an AEAD encryption key ek for encrypting the group data structure, a seed rrs for deriving re-randomized pseudonym keys for this group, and a PKE keypair (ipk, isk) used for encrypting and decrypting join requests.

3.1 Protocols

All protocols, except for user registration, are assumed to run over a server(-only)-authenticated secure channel, i.e., we assume the server is authenticated, but the client is anonymous, and the channel is encrypted. For user registration, we assume a mutually authenticated channel.⁴

User registration (RegisterUser). The user registration protocol `RegisterUser`, shown in Figure 3, registers a new user with the server. The user identifier UID can be thought of as the user’s phone number, under which the user can authenticate to the server. To start registration, the user generates two pairs of signing keys: (vk^{pers}, sk^{pers}) for a regular signature scheme Σ and (vk_{base}, sk_{base}) for a key-rerandomizable signature scheme Π .⁵ The user also samples a main user key muk , from which they then derive their symmetric key material following the key hierarchy (see Figure 2) for their initial epoch ($ep = 1$). The acquaintance tag acq and vk_{base} are then signed with sk^{pers} . The user deposits their acquaintance tag acq , and the signature with the server. This message is transmitted over a UID -authenticated channel.

⁴Signal, for example, authenticates users via an SMS challenge upon registration.

⁵We assume vk^{pers} is available to all users via some PKI ($\mathcal{F}_{PKI}$); in practice Signal’s user verification keys can be reused here.

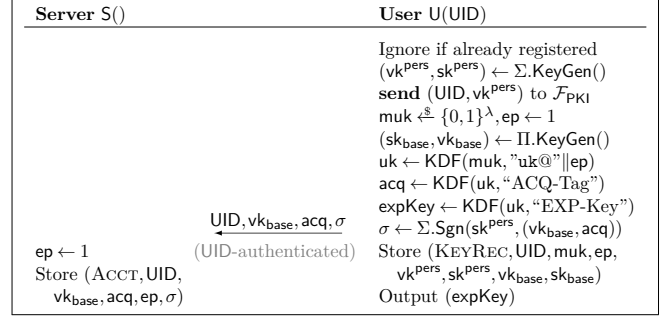


Figure 3: User registration protocol `RegisterUser`.

Granting acquaintance (GrantAcquaintance). A user U grants acquaintance (via the `GrantAcquaintance` protocol, Figure 4) to another user U' by sending them their user identifier UID and key uk . Alternatively, U' might obtain those from the data structure of a group where both are members. U' can validate this information by (anonymously) sending UID to the server and obtaining the rerandomization base vk_{base} and acquaintance tag acq for U from the server, which it locally checks before storing U as acquaintance.

Group creation (Create). Any user can create a new group via the `Create` protocol shown in Figure 5, which initializes a group data structure (see Section 2.2) holding that user as sole member, with admin privileges. The protocol works as follows, for a user with identifier UID , group description $attr$, and group settings b_{attr} (whether or not a user can change the group description) and b_{add} (whether or not users can add more users):

- The server picks the next available group id GID .
- The group creator samples a group key gk and a fetching token fet . These keys will protect the privacy of the group.
- The group creator composes the **header** $hdr \leftarrow (GID, vMaj, c_{hdr}, sttgs)$ as in Figure 5. The sensitive information in the header, the group description $attr$ and settings (b_{attr}, b_{add}) , is encrypted under a the encryption key ek derived from gk .
- The group creator composes the **member list** $mem_lst := (C_{UID}, mct_1, admn = true)$ as specified in Figure 5. While our PGS does not further use the commitment C_{UID} , with the commitment randomness stored in mct_1 , these commitments can be used for liveness checks (we discuss this in Section 5). The user-related private information, such as UID and the user’s key material uk , is encrypted in mct_1 , hiding it from the server.
- The group creator signs the group structure with their pseudonym signing key sk^{psdn} , re-randomized from the creator’s base value sk_{base} using random-

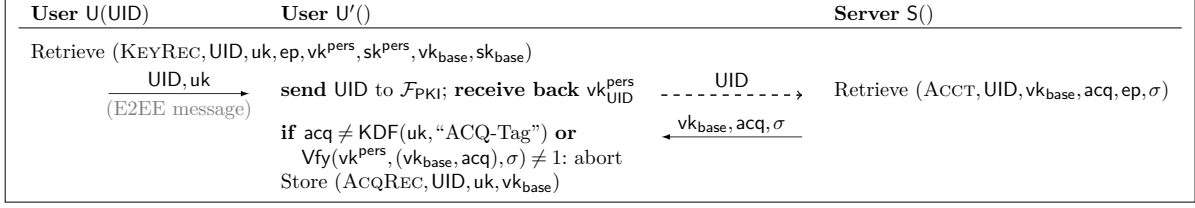


Figure 4: Protocol GrantAcquaintance where user U with user identifier UID allows user U' to add them to groups, and U' validates the acquaintance through interaction with the server. Dashed arrows denote a user-anonymous communication.

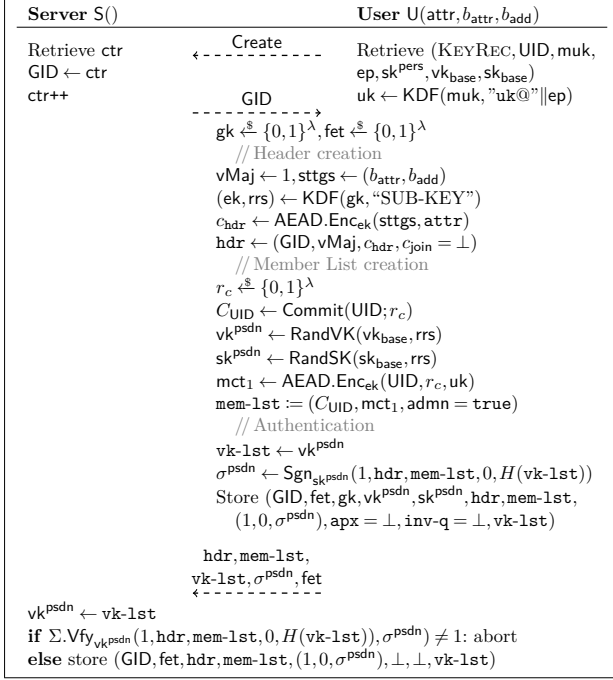


Figure 5: Our protocol Create for creating a group. Dashed arrows denote user-anonymous communication.

ness rrs derived from gk . This allows the server and group members to verify that the data structure was edited by the owner of pseudonym vk^{psdn} , which was tied to user UID upon registration.

- The group data structure (including the header, the member list, the pseudonym verification key and the core signature) as well as the fetching token are sent to the server. The server checks the pseudonym signature and, if it verifies, stores the group data structure for group id GID.

Fetching a group (Fetch). Fetching a group with id GID is shown in Figure 6.

- The user presents GID and the current fetching token fet to the server
- If the token matches the stored token for GID, the

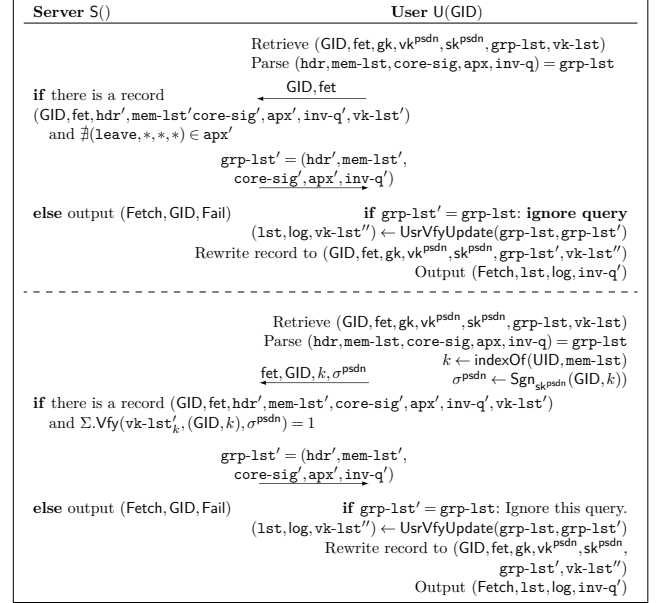


Figure 6: Fetch protocol. The upper part shows fetching with a token (only possible before **leave**). The lower part shows fetching with a pseudonym signature (required after **leave**). After obtaining, the user verifies the update with Figure 10.

server sends the group description to the user.

- The user can check consistency of all entries in the member list, done via a subroutine $UsrVfyUpdate$ (given in Figure 10 in Section 7.1). Essentially, if any mal-formed entry is found, the user can detect this and act accordingly (which, in practice, could mean trying to recover from the error, or as a last resort deleting the whole group). As a result, all honest users have the same view of the group.
- Consistency checking includes:
 - Verifying the header (group id, version numbers progress correctly, flags decrypt).
 - Verifying the admin pseudonym signature, incl all users pseudonym verification keys.
 - Decrypting all member ciphertexts using ek

without an error.

- Verifying acquaintance tags of new members.
- Recomputing and checking the user id commitment for all members.
- Checking the appendix header (group id, version).
- Verifying all appendix entries' signatures and processing the operations.

Note that admins and users should fetch the protocol after any modification to the group list they performed. This is to ensure that the server indeed accepted their modifications.

Adding someone to a group (AddMember). Adding works very similar to group creation, just that now an additional user entry is added. The server checks the executing user is and admin, verifies that admin's pseudonym signature, and adds the new members pseudonym verification key to $\mathbf{vk}\text{-}\mathbf{lst}$. The group key $\mathbf{gk}$ and $\mathbf{fet}$ are shared with the new group member via 1:1 messages. The protocol is depicted in Figure 7.

Removing users (RemoveMember). The protocol is similar to **AddMember**; we hence defer the corresponding Figure 3 to Section 7.3. The only difference is the way that the server composes the new member list $\mathbf{mem}\text{-}\mathbf{lst}$ and pseudonym key list $\mathbf{vk}\text{-}\mathbf{lst}$, namely by removing the j -th entry instead of appending a new one. Note that the protocol also works for an admin removing themselves.

Modifying the appendix. Non-admin members of the group cannot follow the same procedure as admins since they cannot provide an admin signature. To enable members to still perform non-admin group actions, the group structure contains an append-only appendix that collects group structure updates written by non-admins. There, each added entry is signed individually the executing member's pseudonym key.

All non-admin initiated protocols work similarly, and hence we can present them all in Figure 8. The user composes the appendix line $\mathbf{line}$ consisting of the action label, its own index in the membership list, and possibly further encrypted details depending on the operation. When new users join the group, either on their own behalf via **JoinViaLink** or when users add other users via **UserAdd**, the corresponding UID commitment has to be added to the line, and the new joiner's pseudonym verification key has to be computed and communicated to the server. Note that for joining, knowledge of the group key is required, and we propose in Section 7.7 a mechanism for allowing users to derive the group key from a join link. Finally, the user computes a signature of the update under their own pseudonym key. The server obtains

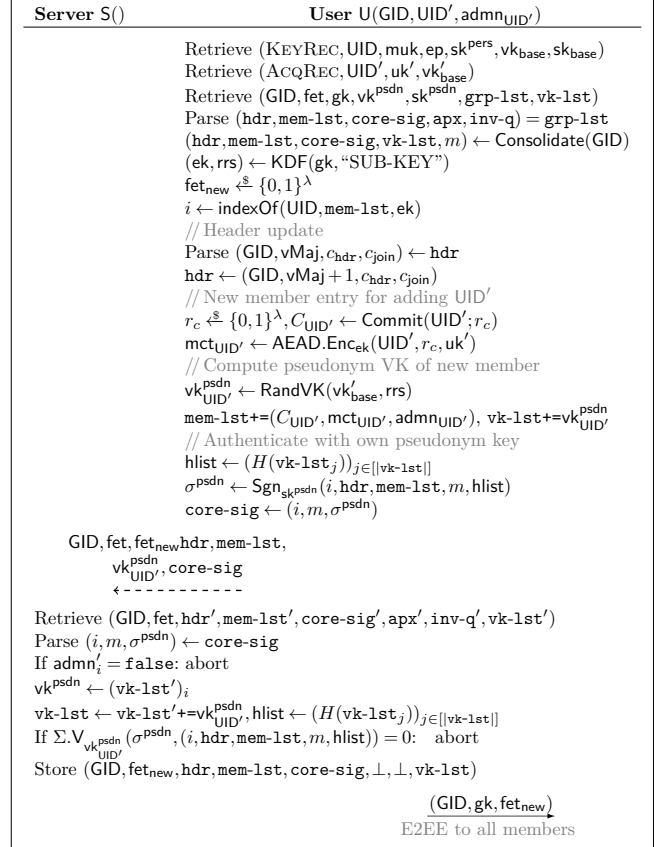


Figure 7: Protocol **AddMember** for adding a new member UID' to a group. $\text{indexOf}(\text{UID}, \text{mem-lst})$ returns the index (row) of UID in mem-lst . Dashed arrows denote user-anonymous communication.

the new line, verifies the signature (either against the verification key stored in the membership list, or against the received ones), and stores the updated appendix. We note that a malicious computation of pseudonym verification keys cannot be detected by the server but will be detected by honest users upon fetching the group state. Such invalid entries will simply be discarded by users (see **Fetch** in Figure 6).

Admins cleaning up the appendix: subroutine (Consolidate). When admin fetches a non-empty appendix, they consolidate all updates into a new major version of the group data structure. For that, the admin first evaluates validity of each appendix line, and writes every valid change to the main list. The protocol is depicted in Figure 9.

Further protocols. Due to space limitations, we defer the description of all further protocols (incl. setting admin rights, changing group settings, inviting users,



Figure 8: Protocols Leave, Refresh, UserAdd, ChangeAttr, JoinViaLink for user U with identifier UID writing into the appendix. Blue lines mark code that is run only in specific protocols. cmd stands for the protocol that is run. The lower part shows the JoinViaLink protocol.

handling join links, and key rotation) to Section 7.

4 Security of Private Group Systems

In this section, we summarize our formal security model for a private group system and the security guarantees. As in previous work [CPZ20], we use the Universal Composability (UC) [Can01] framework for this formalization. Due to the differences in our and their system, we depart significantly from the security model of [CPZ20]. For the reader familiar with the model of [CPZ20] we provide some justification and comparison in Section 4.1.

On a very high level, we define an ideal functionality $\mathcal{F}_{\text{PGS}}$ that serves as a trusted storage for all of the private groups' state. This includes the group metadata (identi-

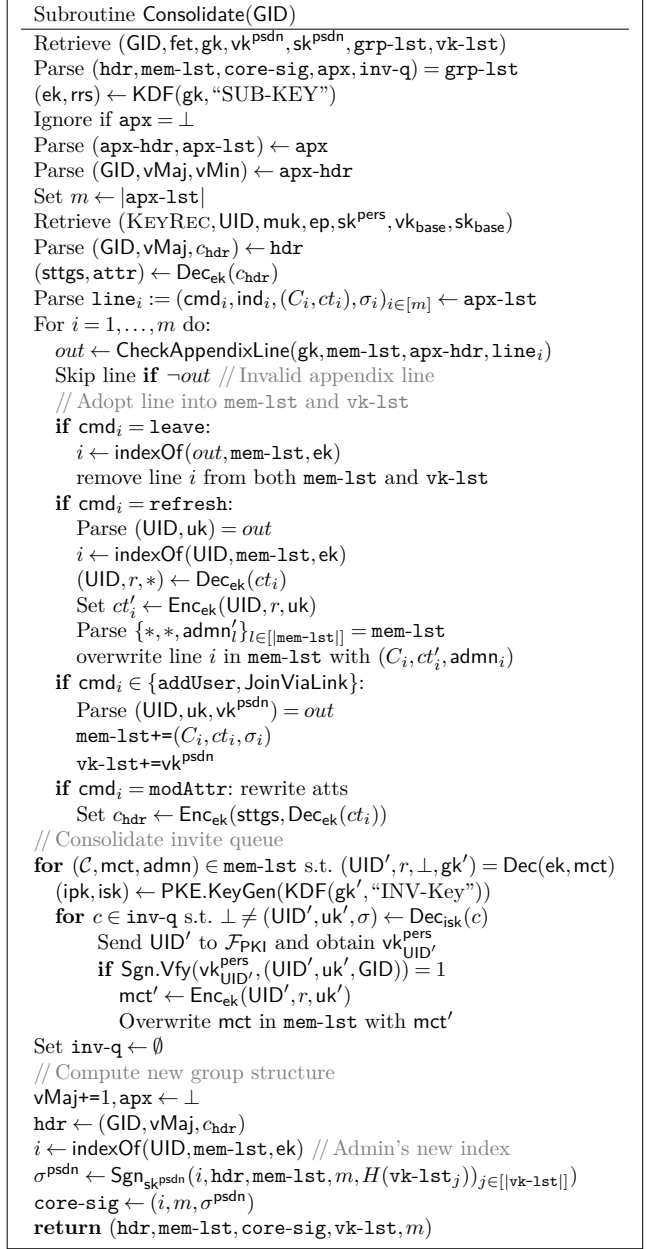


Figure 9: Subroutine Consolidate(GID), run by admins upon fetching GID with a non-empty appendix. The subroutine consolidates the appendix of a stored GID record into its main list mem-1st, proceeding from the oldest to the newest appendix line, and ignoring all invalid lines. The CheckAppendixLine subroutine is defined in Figure 11 in Section 7.2.

fier, attributes, settings), the member list (incl. admin privileges), invites, export key material of all users, as well as a changelog of past group modifications. The core security properties targeted by a private group system are captured by $\mathcal{F}_{\text{PGS}}$ as follows:

- $\mathcal{F}_{\text{PGS}}$ allows access to this state only to group members (capturing *privacy of group memberships*),
- $\mathcal{F}_{\text{PGS}}$ displays the same group state to all members (capturing *consistency*),
- $\mathcal{F}_{\text{PGS}}$ allows manipulations of the state only according to the group settings, without leaking which user made the modification to the server (capturing *anonymous group management*),
- $\mathcal{F}_{\text{PGS}}$ hands export key material only to users that share a group (capturing *confidentiality of key material*), and
- $\mathcal{F}_{\text{PGS}}$ enforces that only acquainted users can be added to a group (capturing *authorized addition*).

Here, the functionality reflects what is currently implemented by Signal. For example, by default, only admins can add, invite, and remove users, and everybody can always leave the group. But $\mathcal{F}_{\text{PGS}}$ also allows admins to configure groups such that, e.g., a group can be joined via a link, users can change the group attributes, or users are allowed to add other users.

Due to space reasons, we only sketch the functionality's behavior for some example protocols, `Create`, `AddMember`, `RemoveMember`, and `UserAdd`; the full formalism is deferred to Section 8.

Group creation. To create a group, any previously registered user calls a `Create` interface, providing group attributes `attr` and initial settings b_{attr} (whether users can change the group attributes) and b_{add} (whether users are allowed to add more users). $\mathcal{F}_{\text{PGS}}$ stores the new group in an object $T_{\text{G}}[\text{GID}][1]$, where 1 indicates the version of the group. The group gets assigned a unique identifier `GID`.

Adding and removing users. After the creation of a group, $\mathcal{F}_{\text{PGS}}$ allows admins to add and eventually remove group members via interfaces `AddMember` and `RemoveMember`. Every such modification of the group state increases the version number, and $\mathcal{F}_{\text{PGS}}$ will maintain all versions of a group. This is done by first copying the whole group state into $T_{\text{G}}[\text{GID}][\text{ver} + 1]$ and then performing the action in the list of members $T_{\text{G}}[\text{GID}][\text{ver} + 1].\text{members}$. $\mathcal{F}_{\text{PGS}}$ enforces that these actions can only be performed by admins by ensuring that the calling user's `UID` is in the `admins` list of the current group version $T_{\text{G}}[\text{GID}][\text{ver}]$.

Non-admin adding of group members is done via a `UserAdd` interface. This interface works very similarly to the `AddMember` interface but allows the request only if the group settings allow users to add members.

Leakage. The ideal functionality formalizes the leakage that a private group system has. While we demand strong privacy properties against non-members and the

server storing the group data, we cannot expect to protect a group's privacy in the presence of corrupt group members. Hence, $\mathcal{F}_{\text{PGS}}$ has several levels of leakage depending on server and member corruption. In a nutshell:

- If the server and all group members are honest, only the type of query is leaked to the adversary (accounting for leakage through communication size and patterns).
- For a corrupt server but honest group members, $\mathcal{F}_{\text{PGS}}$ additionally leaks the touched group's identifier `GID` and the indices of the involved users to the adversary. The group id is a mere administrative handle. The indices of the involved users (e.g., "The 3rd user who joined removed the user who joined 8th") are an artifact of our (as well as Signal's current) protocol that stores an encrypted entry per user. Crucially, these indices do not reveal the identity of the involved users, nor do they allow for linking users across groups.
- When a group member is corrupt, $\mathcal{F}_{\text{PGS}}$ reveals the full query, including involved `UIDs`, to the adversary. Since group changes are visible to all members of the group, this leakage is unavoidable.

Any non-leakage remains protected. Crucially, this means $\mathcal{F}_{\text{PGS}}$ guarantees privacy of group membership (i.e., `UIDs`) towards a corrupt server.

4.1 Comparison with and Remarks on the Model of [CPZ20]

Chase et al. provide the first formal security model for private group systems. Their functionality however, deviates significantly from ours. The most notable design difference is that [CPZ20] provide two sets of interfaces, one set for the case of an honest server, and one set for the case of a corrupt server. Our $\mathcal{F}_{\text{PGS}}$ only has one set of interfaces whose behavior changes depending on the server corruption state. For $\mathcal{F}_{\text{PGS}}$, this works well because the difference between what $\mathcal{F}_{\text{PGS}}$ guarantees in the presence of a corrupt and an honest server is relatively small, while in [CPZ20] the gap is larger.

We now list several aspects and reasons why we chose to re-design a model for PGS from the ground-up.

- **Admin privileges.** Chase et al. do not incorporate different member roles in their model, but allow members to perform any change of the group. Our $\mathcal{F}_{\text{PGS}}$ models admin privileges and flexible configuration of what non-admin users can do to the group.
- **Corrupt server guarantees.** If the server is corrupt, the functionality of Chase et al. does not store any group information anymore, but instead lets the adversary determine the output towards the user. This is a weaker model than our $\mathcal{F}_{\text{PGS}}$, where

a corrupt server alone cannot alter the group state. The only power a corrupt server has in our model is to withhold the latest updates from users who fetch the group. This strengthening is important as one would not want a corrupt server to, e.g., add itself to arbitrary groups, as this would immediately void all privacy guarantees of the PGS.

- **Export/profile key usage.** Chase et al. exploit this user-specific key material to prove acquaintance between users, while in our model, the export key is not used for the PGS but is merely a key that is exported and can be used in arbitrary ways by the application.
- **Key rotation.** For privacy purposes, both group and user keys can be rotated/refreshed in our system, properties that Chase et al. do not model.

4.2 Security Analysis

We provide a security analysis of our private group system in Section 10. In brief, we argue how our design realizes the ideal $\mathcal{F}_{\text{PGS}}$ functionality describing corresponding simulators. For this, we model hash functions and KDFs as random oracles (RO) and the authenticated encryption scheme as ideal authenticated cipher (cf. [JKX18]); further, we assume the commitment scheme to be equivocal and extractable, the signature to be unforgeable, and the key rerandomizable signature scheme to be unforgeability under re-randomized keys (cf. [FKM⁺16, Def. 8]).

5 Deployment Considerations

5.1 Migration

Migrating from one system to another in real time requires having both operational and making sure changes are synchronized between the two. With the design we present here, the only actors that can write to the new system are users, so they will be required to do the synchronization. To accomplish this, non-administrator users will attempt to write to both groups for every update during the transition period. Administrator users will be responsible for reconciling any differences between the two systems and correcting them in the new system. Signal administrators will be able to track when all users have the code installed to handle the new system, and can update code to only use the new system once that can be done safely.

Future migration. While the initial migration from Signal’s current system to our proposed system will be challenging, this system has been designed to make future transitions toward full quantum safety straightforward.

An initial deployment can use efficient primitives with HNDL confidentiality and classical authentication. When quantum-safe authentication becomes a more pressing concern, our system is designed so that each authentication component has a reasonably efficient quantum-safe alternative. Signatures can be replaced with standardized quantum-safe signatures. Commitments, credentials, and zero-knowledge proofs have feasible lattice-based analogues (unlike the verifiable encryption used by Signal today). The Discrete Log-based authorization used for adding pseudonym keys has a Dilithium-based replacement. In each case, there is a clear path forward that will not require transition or redesign of the full system.

5.2 Message Delivery and Sending

This work focused on metadata protection for group management, but metadata can also be leaked by message delivery. Signal’s Sealed Sender protocol protects the social graph by hiding sender identity from the server, but requires authorization tokens or anonymous credentials to prevent abuse. The credential system Signal uses for this today, Group Send Endorsements [Sig25], does not have a feasible quantum-safe analogue, so we must rely on symmetric-key based tokens. In our system, these send tokens can be derived from the exported application key material (`expKey`), which in turn is derived from each user’s key `uk`. Since our model guarantees that only group members learn other members’ latest user keys, this provides a natural authorization mechanism.

However, Sender Keys—Signal’s current approach for efficient group messaging—is fundamentally at odds with social graph protection, as it requires revealing group membership structure to the server. An ideal system would use existing 1:1 channels with Sealed Sender or Anonymity Wrapper [TRB⁺25] for message delivery, potentially with timing jitter to obscure message patterns. Today, for usability and scalability, Signal makes a compromise using Sender Keys with a variant of Sealed Sender, trusting the server not to log delivery patterns. Improvements to this trade-off would be an interesting direction for future work.

5.3 Handling Inactive Users

If all members of a group deregister their accounts, the group becomes unusable. Server administrators want the ability to delete such groups, but due to the metadata protection built into the private group system, they have no way of identifying them. This is a problem in Signal’s system today.

We propose two broad strategies for mitigating this. The first strategy ensures that the group is populated by

registered users of the service, but leaks some information to the server about which group members are active (information that is leaked by Signal’s service today). The second leaks less information and is more lightweight, but offers no assurance that group members are still registered with the service. A mix of both approaches is possible and allows for a tunable trade-off.

Credential-based check-in. We can have clients regularly (e.g., every 30 days) check-in by authenticating against their user identifier commitment and have the server record the time of check-in at low granularity. This ensures not only that the group member is active, but also that they have a valid credential and thus they are, at least recently, a registered user of the service. The server can store the last check-in time per user, per group, or store “last administrator check-in” and “last member check-in” separately. The service can then use this information to set a policy for group maintenance. For example, they could set a policy that any group with no check-in in the last 90 days will be deleted. Similarly, administrators may wish to delete groups where no administrators have been active for an extended time period, as this system requires active administrators to clean up the appendix. With more fine-grained information about last check-in time, informative warnings could be presented to group members.

Token- or signature-based check-in. More lightweight check-ins can be performed using the symmetric key primitives already used for most authorization in this system. In fact, each action, such as fetching a group, can be recorded as a check-in event. This would have many of the features of the credential-based check-in above, but provides no assurance that group participants are still registered users of the service. They could, e.g., build a third-party application using the service infrastructure without registered accounts.

5.4 User Re-registration

While authentication through rerandomizable signatures will work for the large majority of cases, there is an important scenario where a different approach is needed: user re-registration. This can happen when, e.g., a user loses their phone and recovers their account on a new device. The user will not have any cryptographic keys from the old device and will register new keys with the service. At this point, that user will be unable to authenticate with our system. To handle this scenario, anonymous credentials can be used in combination with the user id commitments to allow a client to prove to the server that it holds a credential issued by the server corresponding to a line in the group roster, allowing it to

perform a **refresh** operation. Further, it could encrypt a certificate from the server that its new long-term keys are correctly registered for its user ID in the **refresh** line it adds to the appendix.

5.5 Abuse Mitigation

Several forms of abusive behavior are possible in any cloud-based group system. We discuss how our design addresses these concerns.

Rate-limiting. Signal already has an anonymous credential-based system for rate-limiting resource creation. Clients connect over an authenticated connection to obtain a credential authorizing them to perform an action, such as creating a call link. Later, they connect over an unauthenticated connection, present the credential, and perform the action. We can use this mechanism to limit group creation and to limit non-administrator additions to the appendix.

Storage abuse. The appendix might be abused as free cloud storage. However, the storage capacity available per group is quite small (esp. when comparing to, e.g., message attachments), making this economically unattractive, particularly if gated by a rate-limiting process. A conceivable option to control this directly would be to place a strict cap on the size of an appendix. This would, however, have limited effect, since an attacker could simply create multiple groups. It would also have a clear negative effect on usability, since in groups with a full appendix, non-admin users would be unable to perform actions until an administrator cleans up the group state.

Denial of service and spam. Malicious group members could attempt to spam the appendix with nonsense entries or flood the system with requests. Our design ensures that users can reliably identify which group member is misbehaving (through authenticated appendix entries). This allows the development of product features to enable community members to take appropriate measures—such as notifying users that their clients are malfunctioning or removing problematic members from the group.

5.6 Application Level Features

Some features are better implemented at the application level rather than within the group system.

Blocking a group. Signal’s current implementation handles group blocking outside the cryptographic group system. When a user blocks a group, the client leaves the group, sets a local flag to ignore future messages for that

group, and rotates the user’s profile key. This general approach remains appropriate for our system, with users rotating their uk after leaving the group.

Verifying registration. Checking account existence (via credentials and the user identifier commitments) allows the server to verify that group members are registered with the service, and as described above, this can be used to identify inactive groups or groups with inactive administrators. While our protocol provides this affordance, it is an application level decision when and how this should be used.

References

- [AG24] Martin R. Albrecht and Kamil Doruk Gür. Verifiable oblivious pseudorandom functions from lattices: Practical-ish and threshold-isable. In Kai-Min Chung and Yu Sasaki, editors, *ASIACRYPT 2024, Part IV*, volume 15487 of *LNCS*, pages 205–237. Springer, Singapore, December 2024.
- [BBC] BBC News. Moxie marlinspike leaves encrypted-messaging app signal. *BBC News*. Accessed 05 October 2025.
- [BDFH25] Ward Beullens, Lucas Dodgson, Sebastian H. Faller, and Julia Hesse. The 2Hash OPRF framework and efficient post-quantum instantiations. In Serge Fehr and Pierre-Alain Fouque, editors, *EUROCRYPT 2025, Part VIII*, volume 15608 of *LNCS*, pages 332–362. Springer, Cham, May 2025.
- [BDK⁺21] Shi Bai, Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, Peter Schwabe, Gregor Seiler, and Damien Stehlé. Crystals-dilithium: Algorithm specifications and supporting documentation (version 3.1). <https://pq-crystals.org/dilithium/data/dilithium-specification-round3-20210208.pdf>, 2021.
- [BLNS23] Jonathan Bootle, Vadim Lyubashevsky, Ngoc Khanh Nguyen, and Alessandro Sorniotti. A framework for practical anonymous credentials from lattices. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part II*, volume 14082 of *LNCS*, pages 384–417. Springer, Cham, August 2023.
- [But13] Vitalik Buterin. Deterministic wallets, their advantages and their understated flaws. <https://bitcoinmagazine.com/technical/deterministic-wallets-advantages-flaw-1385450276>, 2013.
- [Can01] Ran Canetti. Universally composable security: A new paradigm for cryptographic protocols. In *42nd FOCS*, pages 136–145. IEEE Computer Society Press, October 2001.
- [CPZ20] Melissa Chase, Trevor Perrin, and Greg Zaverucha. The Signal private group system and anonymous credentials supporting efficient verifiable encryption. In Jay Ligatti, Xinming Ou, Jonathan Katz, and Giovanni Vigna, editors, *ACM CCS 2020*, pages 1445–1459. ACM Press, November 2020.
- [DFL19] Poulami Das, Sebastian Faust, and Julian Loss. A formal treatment of deterministic wallets. In Lorenzo Cavallaro, Johannes Kinder, XiaoFeng Wang, and Jonathan Katz, editors, *ACM CCS 2019*, pages 651–668. ACM Press, November 2019.
- [FHK⁺20] Pierre-Alain Fouque, Jeffrey Hoffstein, Paul Kirchner, Vadim Lyubashevsky, Thomas Pornin, Thomas Prest, Thomas Ricosset, Gregor Seiler, William Whyte, and Zhenfei Zhang. Falcon: Fast-fourier lattice-based compact signatures over ntru. <https://falcon-sign.info/falcon.pdf>, 2020.
- [FKM⁺16] Nils Fleischhacker, Johannes Krupp, Giulio Malavolta, Jonas Schneider, Dominique Schröder, and Mark Simkin. Efficient unlinkable sanitizable signatures from signatures with re-randomizable keys. In Chen-Mou Cheng, Kai-Min Chung, Giuseppe Persiano, and Bo-Yin Yang, editors, *PKC 2016, Part I*, volume 9614 of *LNCS*, pages 301–330. Springer, Berlin, Heidelberg, March 2016.
- [JKX18] Stanislaw Jarecki, Hugo Krawczyk, and Jiayu Xu. OPAQUE: An asymmetric PAKE protocol secure against pre-computation attacks. In Jesper Buus Nielsen and Vincent Rijmen, editors, *EUROCRYPT 2018, Part III*, volume 10822 of *LNCS*, pages 456–486. Springer, Cham, April / May 2018.
- [LNP22] Vadim Lyubashevsky, Ngoc Khanh Nguyen, and Maxime Plançon. Lattice-based zero-knowledge proofs and applications: Shorter, simpler, and more general. In Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part II*, volume 13508 of *LNCS*, pages 71–101. Springer, Cham, August 2022.
- [Lyu09] Vadim Lyubashevsky. Fiat-Shamir with aborts: Applications to lattice and factoring-based signatures. In Mitsuru Matsui, editor, *ASIACRYPT 2009*, volume 5912 of *LNCS*, pages 598–616. Springer, Berlin, Heidelberg, December 2009.
- [Lyu24] Vadim Lyubashevsky. Basic lattice cryptography: The concepts behind kyber (ml-kem) and dilithium (ml-dsa). <https://eprint.iacr.org/2024/1287.pdf>, 2024.

- [Ole] Jim Oleary. Technology preview: Signal private group system. <https://signal.org/blog/signal-private-group-system/>. Accessed 05 October 2025.
- [Sig25] Signal Messenger, LLC. libsignal: Zero-knowledge credential endorsements implementation. <https://github.com/signalapp/libsignal/blob/main/rust/zkcredential/src/endorsements.rs>, 2025. Rust implementation of endorsements module in the zkcredential library, part of the Signal Protocol cryptographic primitives.
- [TRB⁺25] Lea Thiemt, Paul Rösler, Alexander Binstock, Rolfe Schmidt, and Yevgeniy Dodis. Generic anonymity wrapper for messaging protocols. In Chun-Ying Huang, Jyh-Cheng Chen, Shiuh-Pyng Shieh, David Lie, and Véronique Cortier, editors, *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security, CCS 2025, Taipei, Taiwan, October 13-17, 2025*, pages 350–364. ACM, 2025.
- [Wik13] Bitcoin Wiki. BIP 0032. https://en.bitcoin.it/wiki/BIP_0032, 2013.

6 Further System Details

Instantiating key re-randomizable signatures.

The (folklore) version of the re-randomizable signature scheme built on top of the Schnorr scheme, as described in [FKM⁺16], has its algorithms defined as follows: The Gen algorithm outputs $(x, y = g^x)$ as the secret / public values. The RandSK algorithm outputs $\rho + x$, while the RandVK algorithm outputs $y \cdot g^\rho$. Since $g^{\rho+x} = y \cdot g^\rho$, the outputs of RandSK and RandVK form the secret and public key of a Schnorr signature scheme.

In the quantum-safe scenario, the above can be analogously constructed with a modification of the algorithms in the ML-DSA (Dilithium) signature scheme [BDK⁺21]. For simplicity, in this high level description, we will not go into the details of the Dilithium optimizations. Let B be a matrix of the form $B = [A \mid I]$, where A is randomly generated and I is the identity matrix, that is fixed for the entire protocol. The KeyGen algorithm chooses a low-norm vector x and outputs $(x, y = Bx)$ as the secret / public parameters. The secret key x has exactly the same distribution as in Dilithium. The difference between y and the Dilithium public key is that the latter is rounded in order to make it smaller, whereas we will use the entire y . The RandSK algorithm chooses a random small ρ (from the same distribution as x) and outputs $x' = \rho + x$. The RandVK algorithm uses the random ρ to create $y' = y + B\rho$. Since $B \cdot x' = y'$, x' and y' are a secret key / public key pair for a simplified version of the Dilithium signature without rounding the public key (cf. [Lyu24, Figure 5]). At this point, the RandVK can round the public key in the same way as in Dilithium which would make the resulting public key have exactly the same size.

The main effect that the addition of ρ to the secret key x has on the signing algorithm is that now the coefficients of the new secret key are twice as large as before. If we would like to keep the norm of the outputted signature the same, then doubling the size of the coefficients will need to affect the signing time. Lattice-based signatures, such as Dilithium, use the Fiat-Shamir with aborts paradigm [Lyu09] where a signature attempt gets output as a signature only if the potential output vector has small-enough coefficients; and the attempt is repeated otherwise. Doubling the maximum size of the secret key coefficients ends up squaring the expected number of signing attempts (cf. [BDK⁺21, Equation 5 and Table 2]). This translates into an average of about a 5x slowdown versus the ML-DSA standard. But since the signing operation in ML-DSA is already very fast (less than a millisecond on a standard processor [BDK⁺21, Table 1]), and in our scheme these signatures will only be used when the group structure changes, this should not be a problem in practice.

By the Module-LWE assumption, since $(B, B\rho)$ is indistinguishable from uniform, the outputs of RandVK look uniform as well. Forging a signature implies finding a polynomial vector $\mathbf{z}$ with small coefficients and a polynomial c such that $Bz = y'c$. We can rewrite this as $Bz = (y + B\rho)c$, which implies that $B(z - \rho c) = yc$. The coefficients of $z - \rho c$ are small,⁶ and so this implies solving the same SIS problem as in Dilithium (cf. [Lyu24, Equation 81]). The only thing that would change in the actual Dilithium scheme versus the above description is that we would apply a rounding step to y' as in Dilithium's key generation [BDK⁺21, Figure 4, Line 5] in order to reduce the public key size, and then apply the signing and verification algorithms exactly [BDK⁺21, Figure 4] as in the standard, with the value β being set to double the previous value to accommodate the larger secret key parameters. The security of the scheme would follow by noting that forging a Dilithium signature implies extracting random values as in [BDK⁺21, Equation 12] for the re-randomized version and then proceeding as above to show that one can use this to extract the analogous thing for the static t .

User identifier commitments. The member entry also contains a commitment to the user's identifier. This commitment does not serve a specific purpose in our main protocol, but can be used to implement various mechanisms or policies. In Section 5, we already discussed how these commitments can be used in combination with anonymous credentials to allow users to recover from state/key loss. As another example, one could institute periodic spot checks to make sure the list members are active registered signal users. This could be done similarly to how it is currently implemented, using anonymous credentials that have an expiration date. One could also consider issuing tokens that limit the number of operations a user may perform on the group list if the server observes that some lists generate a lot of server overhead. The important point, however, is that these credentials or tokens are no longer tied to the rest of the system beyond users checking that they correctly commit to the group member's user identifier. This allows migrating to quantum-safe component wise, a possibility that Signal's current private group system does not afford. When deploying quantum-safe commitments, these could be stored hashed in the group data structure to reduce the space overhead. Notably, the commitments have to be statistically hiding to never interfere with the confidentiality of user identifiers.

Possible optimizations for the quantum-safe setting. The only public key operations and outputs in

⁶In fact, because z is smaller than in Dilithium, $z - \rho c$ is the same size as in the original scheme.

our system are those of the key re-randomizable signature scheme. While in the classical setting, the outputs can be as short as 32 bytes, when using the quantum-safe ML-DSA standard, the public keys and signatures can be on the order of several kilobytes. To minimize bandwidth and server storage space, it may be better to only send the server hashes of vk^{psdn} instead of vk^{psdn} . Then, when signing (for the first time) under this key, the signer will have to send the vk^{psdn} together with the signature. The advantage of this approach is that, assuming many users in the list will not be performing any group operations, it is wasteful to send the server many vk^{psdn} s that will probably never be used. This optimization has no disadvantages, and so we would recommend its use in the quantum-safe scenario.

Another possible optimization involves a combination of using a key re-randomizable signature scheme whose signatures the server stores locally, and a more efficient (but non-rerandomizable) signature scheme with a static key whose outputs are stored in the group system. The Falcon signature [FHK⁺20], whose signature outputs are roughly 3x smaller than those of ML-DSA, is one such non-rerandomizable possibility. The output of the Falcon signature, of course, cannot be made public to the server (since it will then be able to trace users between groups), but it could be encrypted under the group key. The idea is to include the Falcon signature in the part that is encrypted under the group key, and sign the full update under the ML-DSA vk^{psdn} . The server would not transmit the ML-DSA signature to the other users, but the other users can still verify the correctness of the update using the Falcon signature. The ML-DSA signature under vk^{psdn} would only get used in the case that the Falcon signature does not properly verify, and the rest of the group members need to know who made this incorrect update. They would then request that the server tells them who it is by showing them the more expensive ML-DSA signature. In this scenario, the server would be locally storing (encryptions) of the Falcon signature in addition to all the ML-DSA signatures, but the ML-DSA signatures would only need to be transmitted in case of “disputes”, which are hopefully rare.

On deniability Compared to Signal’s current system, users lose deniability of group membership. That is, another group member can prove to an outsider that this user was in the group (and that he was active).

In the current system, if a member of the group reveals the pseudonym signatures σ^{psdn} of a user together with vk_{base} and the group key gk , to an outsider, $(\sigma^{psdn}, vk_{base}, gk, grp-lst)$ proves to the outsider that the user was indeed a member of that group. For this, the outsider can check that σ^{psdn} verifies under $RandVK(vk_{base}, H(UID, rrs))$, where rrs is decrypted us-

ing gk . In the old system, a group member can also reveal the group list to an outsider. However, there is no proof that the revealed list is correct and not modified. The verifiable encryption of the user’s ID, as well as the profile key commitment, could have been computed by anybody knowing the user. This is because in the old system, only the server verifies the correctness of the group list by checking the anonymous credentials. Neither the model of [CPZ20] nor our model from Section 4 enforces deniability.

7 Protocol Descriptions, Continued

7.1 Verification Subroutine

The subroutine for verifying the group data structure executed by a user upon fetching a new protocol version is given in Figure 10.

7.2 Checking Appendix Lines

The subroutine for checking lines in the appendix is given in Figure 10.

7.3 Removing a Group Member

Figure 12 depicts the RemoveMember protocol.

7.4 Setting Admin Rights (SetAdminRights)

Any admin can grant or revoke admin privileges to other group members. For this, the admin simply toggles the admin flag `admn` in the corresponding row of the member list `mem-lst`, signs the new list and uploads it to the server, who checks the admin’s signature and privileges. We do not depict this protocol as it is straightforward to derive from other admin protocols such as Figure 7.

7.5 Changing the Group Settings (ChangeSettings)

The protocol is depicted in Figure 13. It works similarly to other admin actions, but this time only the header is updated to encrypt the new group settings. The admin authenticates with their pseudonym signature and uploads the update to the server, which checks the admin flag and the signature before storing the update.

7.6 Inviting Users to the Group

Users can get into a group without being acquainted with one of the current group members in two ways: Via

```

UsrVfyUpdate(grp-1st, grp-1st')
Parse (hdr, mem-1st', core-sig', apx', inv-q') ← grp-1st'
Parse (hdr, mem-1st, core-sig, apx, inv-q) ← grp-1st
(ek, rrs) ← KDF(gk, "SUB-KEY")
Parse (GID, *, vMin) ← apx-hdr
Set members =  $\perp$ 
// Check header
Parse (GID, vMaj', chdr', cjoin') ← hdr'
Parse (GID, vMaj, chdr, cjoin) ← hdr
if vMaj' ≤ vMaj : call HandleError // attempted rollback
if vMaj' > vMaj + 1 : call HandleError // version skipped
(sttgs, attr') ← Decek(chdr)
(sttgs', attr') ← Decek(chdr')
// Check core sig
(i, v, σpsdn) ← core-sig'
if vMaj' > vMaj ∧ v ≠ vMin : call HandleError // min version skipped
for j ∈ [1...n]
  UIDj ← GetUID(j, mem-1st, ek)
  vkbase ← GetPseudoVkBase(UIDj)
  vkjpsdn ← RandVK(vkbase, rrs)
vk-1st ← (H(vkjpsdn))j ∈ [1...n]
m ← (i*, hdr, mem-1st, v, vk-1st)
if Sgn.Vfy(vkjpsdn, m, σpsdn) ≠ 1 : call HandleError
if (mem-1st'[i*].admn
  ∧ mem-1st[indexOf(GetUID(i*, mem-1st', ek'), mem-1st, ek)].admn) :
  call HandleError // User made himself admin
// Check member list
Parse {CUIDj, mctj, admnj}j ∈ [1...n] ← mem-1st
For all j ∈ [1...n]:
  (UIDj, rj, ukj, gk') ← Dec(ek, mctj) // gk' ≠  $\perp$  only for invitees
  if CUIDj ≠ Commit(UIDj; rj) : call HandleError
  if ukj =  $\perp$  :
    // Check if UIDj added himself to inv-q
    ukj ← AcceptedInvite(GID, UIDj, inv-q, gk')
    if ukj ≠  $\perp$  : append (UIDj, ukj) to members
  else append (UIDj, ukj) to members
// Check appendix
Parse (apx-hdr, apx-1st) ← apx'
Parse linej := (cmdj, indj, *, *, *)j ∈ [1...|apx-1st|] ← apx-1st
for all j ∈ [1...|apx-1st|] with cmdj ∈ {join, addUser}:
  if false ≠ (UID, uk, vkpsdn) ← CheckAppendixLine(gk, mem-1st', apx-hdr, linej, sttgs, cjoin)
    append (UIDj, ukj) to members
    vk-1st += vkpsdn
for all j ∈ [1...|apx-1st|] with cmdj = refresh:
  if false ≠ (UID, uk) ← CheckAppendixLine(gk, mem-1st', apx-hdr, linej, apx, cjoin)
    replace the j-th entry in members it by (UID, uk)
for all j ∈ [1...|apx-1st|] with cmdj = modAttr:
  if false ≠ (attr') ← CheckAppendixLine(gk, mem-1st', apx-hdr, linej, sttgs, cjoin)
    replace attr' by attr''
for all j ∈ [1...|apx-1st|] with cmdj = leave:
  if false ≠ (UID) ← CheckAppendixLine(gk, mem-1st', apx-hdr, linej, sttgs, cjoin)
    else remove (UID, *) from members
// Check new acquaintances
for all (UID', uk') ∈ members s.t. no record (AcqREC, UID', uk') exists:
  if !IsCorrectUserKey(UID', uk') : HandleError
// Write changelog
if vMin = 0 : changedBy ← i*
else //  $\exists$  appendix entries:
  (*, k, *, *) ← apx-1st[|apx-1st| - 1]
  changedBy ← k
 $\delta \leftarrow \Delta((attr, members), (attr', members'))$ 
if  $|\delta| > 1$  : call HandleError
log ← (changedBy,  $\delta$ ) // line changedBy did  $\delta$ .
// Derive group structure
Parse members = (UID1, uk1), ..., (UIDn, ukn) for some n
Set expKeyi ← KDF(uki, "EXP-Key") for i ∈ [n]
return (attr', (expKeyi, UIDi)i ∈ [1...n], vk-1st)

Procedure HandleError:
  ignore this version and keep the old state.

Procedure AcceptedInvite(GID, UID, inv-q, gk'):
  (ipk, isk) ← PKE.KeyGen(KDF(gk', "INV-Key"))
  for all c ∈ inv-q
    (σ, (UID', uk', GID')) ← Decisk(c)
    if UID' = UID ∧ GID = GID' :
      Send UID' to  $\mathcal{F}_{PKI}$  and obtain vkUID'pers
      if Sgn.Vfy(vkUID'pers, (UID', uk', GID')) : return uk'
  return  $\perp$ 

Procedure IsCorrectUserKey(UID, uk):
  else send UID to S and  $\mathcal{F}_{PKI}$ 
  receive back (vkbase, acq, σ) and vkpers
  if Vfy(vkpers, (vkbase, acq), σ) ≠ 1 ∨ H(uk) ≠ acq : return false
  else return true

```

Figure 10: The verification procedure `UsrVfyUpdate` executed by a user upon fetching a new protocol version with `Fetch`, cf. Figure 6. We write $\Delta((attr, members), (attr', members'))$ for the difference between the two member list versions, i.e., a new (UID, uk) , changed (UID, uk) , a removed (UID, uk) , or changed `attr`.

```

Subroutine CheckAppendixLine(gk, mem-1st, apx-hdr, line, sttgs, cjoin)

Parse (badd, battr) ← sttgs
Parse (cmd, ind, [C], ct, σpsdn) ← line
(ek, rrs) ← KDF(gk, "SUB-KEY")
// Get pseudonym verification key, verify signature:
if ind ≠  $\perp$  : UID ← GetUID(ind, mem-1st, ek)
else (UID, rc, uk) ← Decek(ct)
vkbase ← GetPseudoVkBase(UID)
vkpsdn ← RandVK(vkbase, rrs)
if Sgn.Vfy(vkpsdn, (apx-hdr, line, σpsdn) ≠ 1 : return false
// Checks for a leave line:
if cmd = leave:
  return (UID)
// Checks for a refresh line:
if cmd = refresh:
  uk ← Decek(ct)
  return (UID, uk)
// Checks for lines with new members:
if cmd ∈ {addUser, JoinViaLink}:
  if cmd = addUser ∧  $\neg b_{add}$  : return false
  if cmd = join ∧ cjoin =  $\perp$  : return false
  (UID, rc, uk) ← Decek(ct)
  if C ≠ Commit(UID, rc) : return false
  else return (UID, uk, vkpsdn)
// Checks for a modAttr line:
if cmd = modAttr:
  if  $\neg b_{attr}$  : return false
  attr ← Decek(ct)
  return (UID, attr)

Subroutine GetPseudoVkBase(UID)

if  $\exists$  record (AcqREC, UID, uk, vkbase) : return vkbase
else send UID to S and  $\mathcal{F}_{PKI}$ 
receive back (vkbase, acq, σ) and vkpers
if Vfy(vkpers, (vkbase, acq), σ) ≠ 1 : return Fail
else return vkbase

Subroutine GetUID(i, mem-1st, ek)

if i ≤ |mem-1st| return false
parse {CUIDj, mctj, admnj}j ∈ [1...n] = mem-1st
(UID, *, *) ← Decek(mcti)
return UID

Subroutine indexOf(UID, mem-1st, ek)

parse {CUIDj, mctj, admnj}j ∈ [1...n] = mem-1st
find i ∈ [n] s.t. (UID, *, *) ← Decek(mcti)
return i

```

Figure 11: Subroutines called by our protocols: `CheckAppendixLine(gk, mem-1st, apx-hdr, line)` for checking validity of an appendix line `line` against `mem-1st`. `GetPseudoVkBase(UID)` returns the base pseudonym verification key of `UID`. `GetUID(i, mem-1st, ek)` returns the user identifier encrypted under `ek` in the i -th entry of `mem-1st`. `indexOf(UID, mem-1st, ek)` returns the index of `UID` in the `ek`-encrypted `mem-1st`.

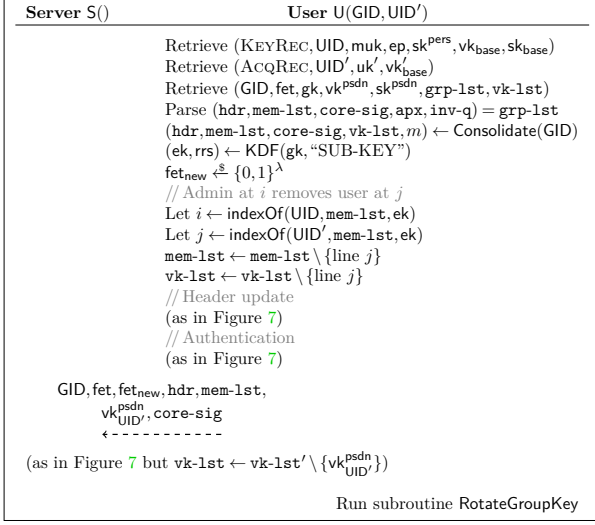


Figure 12: Protocol RemoveMember where user U with identifier UID removes the user with identifier UID'. indexOf(UID, mem-1st) returns the index (row) of UID in mem-1st. Dashed arrows denote user-anonymous communication.

a join link and via an invitation. We start by describing the invitation procedure.

An admin who only knows the UID but not the user key uk of another user (i.e., only knows their phone number but never received a message from them) can invite the latter to the group. This user can decide whether they accept or decline the invitation.

To issue an invitation, an admin generates a key pair $(ipk, isk) \leftarrow \text{PKE.KeyGen}(\text{KDF}(\text{gk}, \text{"INV-Key"}))$. It then performs the code of protocol AddMember (cf. Figure 7) but adding line $(C_{\text{UID}}, \text{Enc}(\text{ek}, \text{UID}, r, \perp, \text{gk}), 0)$ to the main list mem-1st and deriving $\text{vk}_{\text{UID}}^{\text{psdn}}$ not from a stored record but from the server by calling GetPseudoVkBase(UID). The admin then signs and propagates this new group list to the server.

The next step of the invitation process is that the user accepts the invitation. The user first signs its uk as $\sigma^{\text{join}} \leftarrow \Sigma.\text{Sgn}(\text{sk}^{\text{pers}}, (\text{UID}, \text{uk}, \text{GID}))$ and then encrypts it together with the signature $c \leftarrow \text{PKE.Enc}_{\text{ipk}}(\text{UID}, \text{uk}, \sigma)$ under the received ipk. Ciphertext c is then uploaded to the server, which adds it to the invite queue inv-q. Not accepting an invitation works by simply not acting on the invite. The moment a user key uk for an invited UID is found in the invite queue, fetching users consider the invited user as a member (cf. Fetch, Figure 6).

The admin can also revoke the invitation if it was not accepted yet by removing the corresponding $(C_{\text{UID}}, \text{mct}, 0)$ line, similar as in RemoveMember (cf. Figure 12).

When an admin consolidates the group, they empty

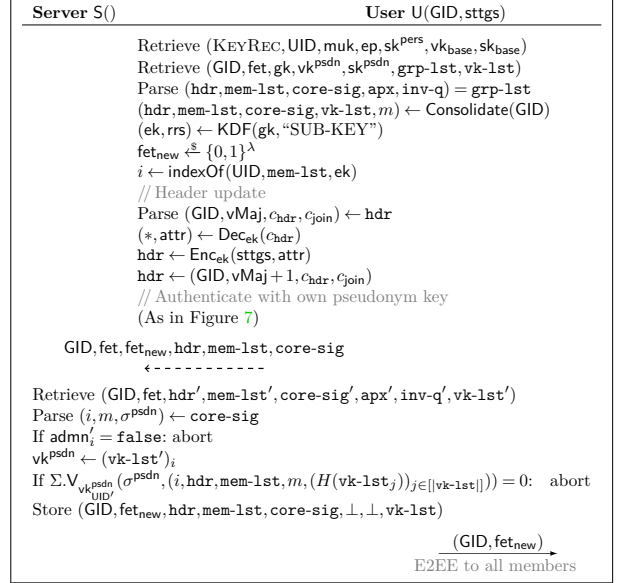


Figure 13: Protocol ChangeSettings for an admin to change the group settings that are encrypted in the header hdr of the group data structure.

the invite queue. That is, they take all entries in the group list that do not contain a userkey, i.e., $\text{uk} = \perp$, and check if there is a corresponding ciphertext $c \in \text{inv-q}$ such that c decrypts to the same UID and the signature in c verifies under UID's public key. In that case, the admin write the uk found in c into the main list. If there are ciphertexts in inv-q without a matching entry in the main list, the admin can discard them. Note that the admin can derive all required gks that are used to derive isk from the group list entries.

7.7 Public Groups: Joining via a Link

Admins can create a link that allows users to join the group without being acquainted with a member of the group. For usability, this link must not change even when the group key is rotated. When a user clicked the link, still either an admin or a user (depending on the group setting) has to complete the join request. Joining via the link can further be configured to require admin approval.

We can realize such links by allowing users to write join entries in the appendix. This was described already in Figure 8. We now propose a mechanism to let users learn ek from a link. Note that we cannot simply encode ek and fet in the link, since otherwise every rotation of the group key (executed at least every time a user is removed from the group) would invalidate the link. Hence, we propose to encode a "join key" jk in the link instead, and let the server store the current group key encrypted under this jk in the group header. When a user

wants to join via a link, it sends GID to the server. If the server finds a non-empty join ciphertext in the header, it sends it back to the user, who decrypts the group key and fetching token, fetches the group structure with the token, and then joins the group via the appendix.

The admin who creates the join link also needs to specify via a boolean b_{adm} whether admins need or need not approve joiners from links. This configuration is uploaded to the server by `ChangeSettings` (cf. Figure 13). Users act on it upon fetching as follows: If the header contains a join ciphertext and $\neq b_{\text{adm}}$, they interpret all correctly, decrypting and verifying join entries in the appendix as members.

Upon group key rotation, admins encrypt the new key material under the static join key jk . For this, the admin who generated the join link has to communicate jk to all other admins (cf. Figure 14).

7.8 Changing the Group Key: Subroutine (RotateGroupKey)

To rotate the group key material, an admin chooses a new group key gk and encrypts the whole group data structure under this new key. The protocol is depicted in Figure 14.

8 Security Model for PGS

In this section, we formally define a private group system and its security guarantees. As in previous works [CPZ20], we use the Universal Composability (UC) [Can01] framework. Due to the differences in our and their system, we depart significantly from their security model. For the reader familiar with the model of [CPZ20] we provide some justification and comparison in Section 4.1. Our formal model is underpinned by a systematization of system properties and guarantees, which we summarize in Section 9.

On a very high level, functionality $\mathcal{F}_{\text{PGS}}$ serves as a trusted storage for all of the private groups' state. A group's state consists of

- group identifier, attributes, and settings,
- current group members,
- current group admins,
- invited users who have not yet accepted the invite,
- export keys of all members, and
- changelog of all past group modifications.

$\mathcal{F}_{\text{PGS}}$ allows access to this state only to group members, and it allows to manipulate the state according to the group settings. Here, the functions offered by $\mathcal{F}_{\text{PGS}}$ reflect what is currently implemented by Signal. For example, by default, only admins can add, invite, and remove users, and everybody can always leave the group. But $\mathcal{F}_{\text{PGS}}$ also allows admins to configure groups such that, e.g.,

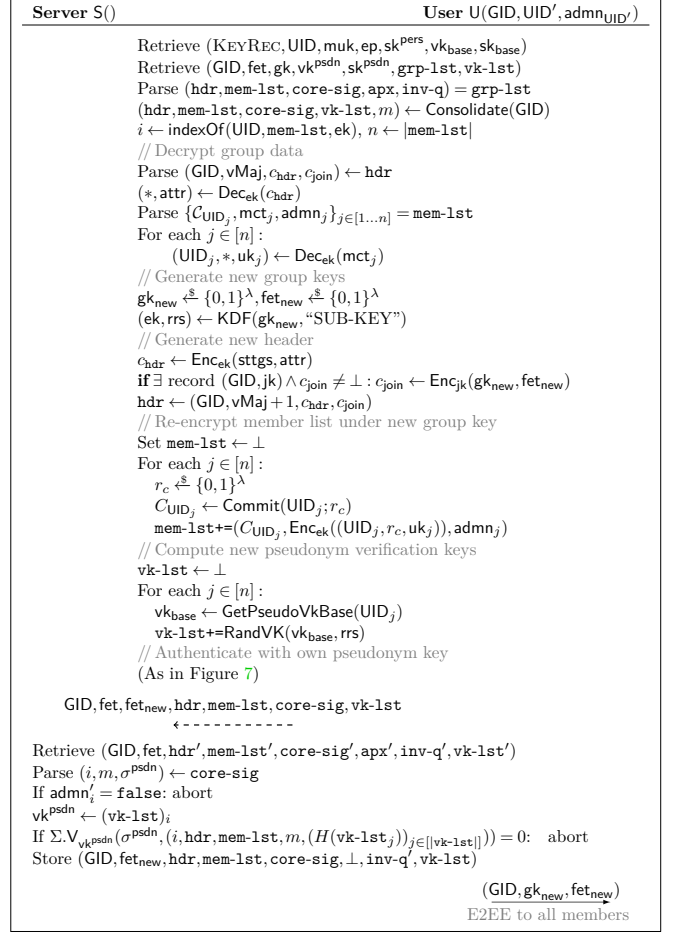


Figure 14: Subroutine `RotateGroupKey` for an admin to rotate the group key.

a group can be joined via a link, users can change the group attributes, or users are allowed to add other users.

USER REGISTRATION. The functionality requires users to register (via interface `RegisterUser`, Figure 15) before being allowed to join or create a group. This registration requires setting a user identifier `UID`, a unique identifier under which the user is referred to in the system from now on, and a (system-wide) key `expKey`. The latter is not required for the PGS but is exported to the application, where it can be used, e.g., to encrypt a user's profile image. $\mathcal{F}_{\text{PGS}}$ ensures that a user's `expKey` is only known among its groups, and it allows for rotating this key via the `Refresh` interface (Figure 16). In Signal, to protect users from being added to spam groups, one can only add users to a group from whom one has previously received a message, or with whom one shares a group. $\mathcal{F}_{\text{PGS}}$ reflects the message approval mechanism by letting Bob call `(GrantAcquaintance, Alice)`. Afterwards, Alice can add Bob to any group. $\mathcal{F}_{\text{PGS}}$ stores Alice's acquaintances in list $T_{\text{Acq}}[\text{Alice}]$ (Figure 15). While the

server engages in the approval procedure, it only learns Alice’s identity, while Bob remains anonymous (GA.3). In other words, the PGS leaks to the server how “famous” users are, but not who already talked to whom.

GROUP CREATION AND ADDING/REMOVING USERS. To create a group, any registered user calls the **Create** interface, providing group attributes attr and initial settings b_{attr} (whether users can change the group attributes) and b_{add} (whether users are allowed to add more users). $\mathcal{F}_{\text{PGS}}$ stores the new group in an object $T_G[\text{GID}][1]$, where 1 indicates the version of the group. The group gets assigned a unique identifier GID (CG.2). After the creation of a group, $\mathcal{F}_{\text{PGS}}$ allows admins to add and eventually remove group members via interfaces **AddMember** and **RemoveMember**. Every such modification of the group state increases the version number, and $\mathcal{F}_{\text{PGS}}$ will maintain all versions of a group. This is done by first copying the whole group state into $T_G[\text{GID}][\text{ver} + 1]$ (see AM.3 (a) and RM.3 (a)) and then performing the action in the list of members $T_G[\text{GID}][\text{ver} + 1].\text{members}$. $\mathcal{F}_{\text{PGS}}$ enforces that these actions can only be performed by admins by ensuring that the calling user’s UID is in the admins list of the current group version $T_G[\text{GID}][\text{ver}]$ (AM.3, RM.3).

FETCHING THE GROUP STATE. Any group member can fetch the group state by calling the **Fetch** interface (Figure 15) with the corresponding group identifier GID and the version the user last fetched. $\mathcal{F}_{\text{PGS}}$ ensures that users fetch all updates sequentially (FG.2, FG.3). It also ensures that non-members cannot fetch even when they know the group ID (FG.6). Whenever a member fetches, $\mathcal{F}_{\text{PGS}}$ stores the version presented to the user in table $T_{\text{Ver}}[\text{GID}][\text{UID}]$ (FG.8 (a)), adds the user as acquaintance to all group members (FG.8 (b), meaning the user can from now on add all of them to more groups), and finally outputs the group state. This state consists of the identifiers and export keys of all group members, the fetched version number, the group attributes, the list of invited users, and the full changelog since the creation of the group (FG.8 ©). If the server is honest, $\mathcal{F}_{\text{PGS}}$ ensures that users always fetch the latest version.

ADVANCED GROUP FUNCTIONS. $\mathcal{F}_{\text{PGS}}$ offers various interfaces for advanced manipulation of a group state (Figure 16). Since all interfaces follow a similar structure but merely modify different parts of the group state object $T_G[\text{GID}]$, we only provide a brief description of them below. All interfaces have in common that they write successful executions into the group’s changelog.

- **Invite a user to a group (Invite):** for each group, $\mathcal{F}_{\text{PGS}}$ maintains an invite queue $T_G[\text{GID}][\text{ver}].\text{inv-q}$ to which admins can add users of whom they only know the UID , i.e., to whom they are not acquainted. This models inviting users by just typing their phone number. Users in the invite queue are not considered

members yet, but their identifiers are already visible to other group members (“Admin x invited user y to the group”).

- **Revoke a user invitation (RevokeInvitation):** Similarly, admins can remove users from the invite queue.
- **Respond to an invitation (AcceptInvitation):** Invited users only become members if they accept an invite via this interface. $\mathcal{F}_{\text{PGS}}$ then writes them into the member list members .
- **Activate or deactivate a group link (ToggleJoinLink):** For “public” groups that can be joined by users without requiring an invite/add operation from a group member, this interface can be called. To add at least some control over who is joining, admin approval can be activated for users who join on their own behalf. $\mathcal{F}_{\text{PGS}}$ simply stores these settings in two booleans $T_G[\text{GID}][\text{ver}].\text{activeLink}$ and $T_G[\text{GID}][\text{ver}].\text{adminApproval}$.
- **Join a group via a link (JoinViaLink):** If the above interface was used to open up a group, this interface allows users who only know the GID to join the group. In case admin approval is required, $\mathcal{F}_{\text{PGS}}$ will only create a record of a user wanting to join (JvL.4).
- **Admin approval of joining via a link (ApproveJoin):** This interface allows an admin to approve a user who joins a link-activated group. $\mathcal{F}_{\text{PGS}}$ will look for a matching record (AJ.3) and write the user into the members list. We note that, while the names of this and the previous interfaces contain the word “link” as this is how we know them from Signal, $\mathcal{F}_{\text{PGS}}$ never exports or requires such a link: how the “open joining” feature is realized, e.g., how a link concretely looks like is left completely up to the protocol and hence not part of the definition.
- **Changing admin privileges (SetAdminRights):** Any admin can set or unset another member’s admin permissions. $\mathcal{F}_{\text{PGS}}$ reflects this by adding or removing the member in the admins list.
- **Leaving a group (Leave):** Any member can leave the group. $\mathcal{F}_{\text{PGS}}$ denotes the intent of the user to leave by adding the user to a list leaveQueue (LG.3 (c)) and removing them from the members list. The effect of the user remaining in the leave queue is that it is still able to fetch group updates *if* a corrupt server allows them to fetch (FG.6). This accounts for the fact that group key rotation can only happen when another group member initiates it, and hence, the gone user is still able to decrypt data stored at the server. The leave only becomes effective when an admin comes online. $\mathcal{F}_{\text{PGS}}$ reflects this by flushing the leaveQueue in every interface that requires admin privileges (e.g., AM.3 (e)).
- **Non-admin adding of group members (UserAdd):** If

Object	Type	Description
$T_U[\text{UID}]$	object	user data
.owner	pid	UC pid of user who registered UID
.expKey	bits	user's export key
$T_{\text{Ver}}[\text{GID}][\text{UID}]$	int	version of GID that UID last fetched
$T_{\text{Acq}}[\text{UID}]$	list	users who can add UID to a group
$T_G[\text{GID}][i]$	object	i -th group state
.admins	list	UIDs of all group admins
.members	list	UIDs & exp. keys $\forall$ users in group
.log	list	group change since version $i - 1$
.invited	list	UIDs of invited users
.activeLink	bool	invite link activated?
.adminApproval	bool	join via link req. admin approval?
.allowUserAttr	bool	users can change group attributes?
.allowUserAdd	bool	users can add group members?

Table 2: Objects maintained by functionality $\mathcal{F}_{\text{PGS}}$.

the group is configured to allow users to add members, this interface can be used for this operation. It works very similarly to the **AddMember** interface but allows the request only if $T_G[\text{GID}][\text{ver}].\text{allowUserAdd}$ is set (**UA.3**).

- **Changing the group attributes (ChangeAttr):** This interface can be called by any group member. $\mathcal{F}_{\text{PGS}}$ allows to set the group attributes **attr** if the calling user is either an admin, or a member of the group, and the group setting $T_G[\text{GID}][\text{ver}].\text{allowUserAttr}$ is set to **true**.
- **Changing the group settings (ChangeSettings):** This interface allows admins to change the group settings, namely whether users can add other users, and whether they can modify the group attributes. $\mathcal{F}_{\text{PGS}}$ simply checks whether the calling user is an admin in the group (**GS.3**) and writes the settings in the group object $T_G[\text{GID}]$.

This concludes the description of the interfaces of $\mathcal{F}_{\text{PGS}}$. For convenience, we give an overview and short description of all user- and group-specific objects and variables that $\mathcal{F}_{\text{PGS}}$ stores in Table 2.

LEAKAGE: We now turn to describing the leakage that $\mathcal{F}_{\text{PGS}}$ admits. While we demand strong privacy properties against non-members and the Signal server storing the group data, we cannot expect to protect a group's privacy in the presence of corrupt group members. Hence, $\mathcal{F}_{\text{PGS}}$ has several levels of leakage depending on server and member corruption. To simplify the definition, we opt for the same leakage in all group state manipulating interfaces, even though a concrete protocol might provide stronger guarantees for some of them. These leakage levels are (see Figure 15):

- **Honest server & all group members honest (PQ.4):** Only the type of query is leaked to the adversary. This accounts for the fact that commu-

nication size and pattern might already leak which operation is executed, and we do not expect a protocol to hide this information.

- **Corrupt server & all group members honest (PQ.5):** $\mathcal{F}_{\text{PGS}}$ leaks the type of query, the group identifier GID of the group to be modified, and the indices of the involved users to the adversary. A server must know which stored group to update, and group IDs do not contain any private information as they are anyway chosen by the adversary (**CG.1**). The indices of the involved users (e.g., “The 3rd user who joined removed the user who joined 8th”) are an artifact of our protocol that stores an encrypted entry per user. Crucially, these indices do not reveal the identity of the involved users, nor do they allow linking users across groups.
- **At least one group member is corrupt (PQ.6):** $\mathcal{F}_{\text{PGS}}$ reveals the full query, including involved UIDs, to the adversary. Since group changes are visible to all members of the group, this leakage is unavoidable.

This completes the list of information leakage to the adversary, i.e., everything that is not mentioned remains protected. Crucially, $\mathcal{F}_{\text{PGS}}$ guarantees privacy of group membership (i.e., UIDs) towards even a corrupt server.

ATTACKS: Depending on the corruption state, $\mathcal{F}_{\text{PGS}}$ allows a number of attacks. $\mathcal{F}_{\text{PGS}}$ does not contain interfaces that are unlocked if a group has **corrupt members** or even **corrupt admins**. By this, $\mathcal{F}_{\text{PGS}}$ mandates that all such corrupt users can do amounts to *what honest members and admins can do*. In particular, such corrupt users gain no insights or powers over groups they are not a member of. A corrupt admin can upload an invalid group state, in which case $\mathcal{F}_{\text{PGS}}$ ensures that honest users learn who to blame (**FG.6** (b)). Next, we consider a **network adversary** who is present even if all users and the server are honest. A PGS protocol is always prone to Denial-of-Service (DoS) attacks by the network attacker dropping or tampering with messages from or to the server. $\mathcal{F}_{\text{PGS}}$ accounts for this by allowing the adversary to **DENY** all operations (e.g., **AM.2**). To completely suppress the interface call (e.g., dropped first message), the adversary may infinitely delay the **Fail** output to the server. In any such network attack, the group state in $\mathcal{F}_{\text{PGS}}$ will remain the same, i.e., the network adversary is not given the power of impacting the group state beyond preventing updates via the **DENY** flag. Finally, we detail which attacks $\mathcal{F}_{\text{PGS}}$ admits to a **corrupt server**. First, a corrupt server can decide to acquaint arbitrary users, i.e., to always make acquaintance checks pass when a user is added (**GA.1**). A corrupt server can also arbitrarily chose the group identifier GID (**CG.2**), which however

serves only as a pointer. A corrupt server can always “destroy” the group list by tampering with it; however, $\mathcal{F}_{\text{PGS}}$ demands that in such a case, users learn that the server is to blame (FG.7 (a)). A corrupt server can also *withhold group updates* from individual users fetching the group. While this breaks consistency of a group’s state presented to all members, it also seems unavoidable that a server can simply opt to present outdated versions of the group to a fetching user. $\mathcal{F}_{\text{PGS}}$ hence allows the adversary to prevent fetching an update via the **DENY** flag (FG.6). However, $\mathcal{F}_{\text{PGS}}$ protects against *rollback attacks* by demanding the fetched version to be *more recent* than what the user previously fetched ($T_{\text{Ver}}[\text{GID}][\text{UID}]$ stores this information). Additionally, $\mathcal{F}_{\text{PGS}}$ ensures that users do not miss updates (FG.2), i.e., it guarantees that if the user outputs group data, it is the update that happened after the user last fetched. Finally, in the case of a corrupt server colluding with a corrupt group member, $\mathcal{F}_{\text{PGS}}$ does not provide consistency anymore and lets the adversary determine the output of **Fetch** (FG.7 (c)). This reflects the fact that, with all group key material exposed, the server and user can completely rewrite the whole group structure. This concludes the description of the attacks admitted by our model, and hence, we do not expect a PGS protocol to protect against them. Crucially, all attacks not listed in $\mathcal{F}_{\text{PGS}}$ need to be protected against by a PGS protocol. To give some examples, these include a corrupt server manipulating the group state, or a colluding user and server bypassing the group settings.

9 Systematization of System Properties and Guarantees

We provide an easily accessible list of security properties of a PGS scheme and detail in which corruption settings they hold and how they are guaranteed by our protocol. While the formal model in Figures 15 and 16 can be used to derive all these, the list helps the reader interpret the model and understand how our PGS protocol satisfies it. We use the terminology *user* for actual humans using the PGS, and *client* for the user device.

We mark with

- ✓^{P} *positive* properties that are satisfied even when party P behaves maliciously. We assuming all other parties to be semi-honest. If we write ✓^{P,R}, both parties, P and R may be malicious.
- ✗ *negative* properties (i.e., ones that are *not* achieved, leading to *weaker* security or features)

Accepted risk. The following risks are accepted in the system:

- Rogue acquaintances of Alex can give Alex’s uk to an adversary who can then claim to know Alex, send

Alex messages, and read their profile information.

- Rogue group members know the group secret and can share it with anyone.
- Rogue group members know the group list in the clear and can share it with anyone.

10 Security Analysis

We sketch how to simulate each individual interfaces. The case where both parties are corrupted is trivial to simulate, as the simulator just executes all instructions it receive from the environment.

For the sake of this discussion, we will assume that all hash functions and KDFs are modeled as random oracles (RO), the AEAD encryption scheme is modeled as ideal authenticated cipher (cf. [JKX18]), the commitment scheme is assumed to be equivocal and extractable, the signature are assumed to be unforgeable, and the key rerandomizable signature scheme has Unforgeability under Re-randomized Keys (cf. [FKM⁺16, Def. 8]).

Honest server, honest user: For all protocols, simulation is facilitated by our use of secure channels. The simulator merely has to simulate that messages are sent over the network. Because the environment calls the functionality interfaces of the honest parties, the functionality creates all records correctly.

Simulation of RegisterUser *Honest server, honest user:* Additionally to the secure channel, the simulator picks a uniformly random uk and computes acq like in the real protocol as $\text{KDF}(\text{uk}, \text{“ACQ-Tag”})$. Note that the environment receive a uniformly random expKey picked by the functionality and yet unknown to the simulator. If the user gets corrupted at some point later, the simulator can program the RO KDF, such that expKey is the output of $\text{KDF}(\text{uk}, \text{“EXP-Key”})$. The simulator also chooses $(\text{sk}_{\text{base}}, \text{vk}_{\text{base}})$ for this user and stores them.

Honest server, corrupt user: The simulator obtains UID directly from the adversarial message. The simulator observes the RO KDF. If there is a query such that $\text{acq} = \text{KDF}(\text{uk}^*, \text{“ACQ-Tag”})$, then the simulator computes $\text{expKey}^* \leftarrow \text{KDF}(\text{uk}^*, \text{“EXP-Key”})$ and gives input $(\text{RegisterUser}, \text{UID}, \text{expKey}^*)$ to $\mathcal{F}_{\text{PGS}}$ on behalf of the corrupt user.

Corrupt server, honest user: The simulator samples $(\text{sk}_{\text{base}}, \text{vk}_{\text{base}})$, $(\text{sk}^{\text{pers}}, \text{vk}^{\text{pers}})$ like in the real protocol and uk, acq as above. With these values, the simulator computes σ like in the real protocol.

Simulation of Create *Honest server, corrupt user:* In this case, the simulator must extract the secret inputs

The functionality keeps initially empty tables T_U for user-specific data, $T_G[\text{GID}][\text{ver}]$ for the data of group GID in version ver, $T_{\text{Ver}}[\text{GID}]$ for storing the current version of each group, $T_{\text{Ver}}[\text{GID}][\text{UID}]$ for information which version a user last fetched, and $T_{\text{Acq}}[\text{UID}]$ stores the set of users who are acquainted with UID, i.e., who can add UID to a group. We implicitly assume all outputs are *delayed outputs*. The function $\text{indexOf}(\text{UID}, T_G[\text{GID}].\text{members})$ outputs the index of $(\text{UID}, *)$ in the list $T_G[\text{GID}].\text{members}$ (starting at 1). We assume implicit session ids sid in all messages. Initialize $\text{ctr} \leftarrow 1$.

On input (RegisterUser, UID, expKey*) from U:

- R.1 if U already registered: **ignore query**
- R.2 **send** (RegisterUser, UID) to $\mathcal{A}$.
- R.3 if $T_U[\text{UID}]$ is already defined **output** ("bad request") to U and (Error, "user already exists") to S
- R.4 **else** initialize $T_U[\text{UID}]$
- R.5 Set $T_U[\text{UID}].\text{owner} \leftarrow \text{U}$
- R.6 Sample $T_U[\text{UID}].\text{expKey} \leftarrow \{0, 1\}^\lambda$
- R.7 if U is corrupt, set $\text{expKey} \leftarrow \text{expKey}^*$
- R.8 **else output** expKey to U

On input (GrantAcquaintance, UID') from U or $\mathcal{A}$:

- GA.1 if called by $\mathcal{A}$ and S is honest **ignore query**
- GA.2 Find UID s.t. $T_U[\text{UID}].\text{owner} = \text{U}$, **else ignore query**
- GA.3 if S is corrupt, **send** (GrantAcquaintance, UID') to $\mathcal{A}$, **else send** (GrantAcquaintance) to $\mathcal{A}$; **receive back** m
- GA.4 if $T_U[\text{UID}']$ is not defined or $m = \text{DENY}$ **output** (GrantAcquaintance, UID', Fail) to S
- GA.5 **else** add U to $T_{\text{Acq}}[\text{UID}']$

On input (Create, attr, b_{attr} , b_{add}) from U:

- CG.1 **send** (Create) to $\mathcal{A}$; **receive back** (GID, m)
- CG.2 if GID already exist, **ignore query**
- CG.3 if $m = \text{DENY}$, **output** (Create, GID, Fail) to S
- CG.4 if $m = \text{ALLOW}$:
 - (a) find UID s.t. $T_U[\text{UID}].\text{owner} = \text{U}$; **else ignore query**
 - (b) set $T_{\text{Ver}}[\text{GID}] \leftarrow 1$, $T_{\text{Ver}}[\text{GID}][\text{UID}] \leftarrow 1$ and initialize $T_G[\text{GID}][1]$
 - (c) set $T_G[\text{GID}][1].\text{attr} \leftarrow \text{attr}$
 - (d) append UID to $T_G[\text{GID}][1].\text{admins}$
 - (e) append (UID, $T_U[\text{UID}].\text{expKey}$) to $T_G[\text{GID}][1].\text{members}$
 - (f) set $T_G[\text{GID}][1].\text{allowUserAttr} \leftarrow b_{\text{attr}}$
 - (g) set $T_G[\text{GID}][1].\text{allowUserAdd} \leftarrow b_{\text{add}}$
 - (h) set $T_G[\text{GID}][1].\text{log} \leftarrow (\text{UID}, \text{created}, b_{\text{attr}}, b_{\text{add}})$

On input (Fetch, GID) from U:

- FG.1 Find UID s.t. $T_U[\text{UID}].\text{owner} = \text{U}$, **else ignore query**
- FG.2 Set $\text{ver} \leftarrow T_{\text{Ver}}[\text{GID}][\text{UID}] + 1$ // Fetching the next update
- FG.3 Ignore the query if $\text{ver} = T_{\text{Ver}}[\text{GID}]$ // User already up to date
- FG.4 if S honest **then send** (Fetch) to $\mathcal{A}$ **else send** (Fetch, GID) to $\mathcal{A}$
- FG.5 **receive back** m from $\mathcal{A}$
- FG.6 if $m = \text{DENY}$ or $T_G[\text{GID}]$ undefined or $((\text{UID}, *) \notin T_G[\text{GID}][\text{ver}].\text{members} \wedge \neg(\text{UID} \in T_G[\text{GID}][\text{ver}].\text{leaveQueue} \wedge \text{S corrupt}))$, **output** (Fetch, GID, Fail) to S // Left user can still fetch if corrupt Server helps
- FG.7 if $m = (\text{ATTACK}, \text{P}, \text{resp}^*)$:
 - (a) if $(\neg \text{isCorrupt}(\text{GID}) \vee \text{P} = \text{S})$ and S is corrupt, **output** (Fetch, Fail, S) to U // Server to blame
 - (b) if $T_U[\text{P}].\text{owner}$ is corrupt and $\text{P} \in T_G[\text{GID}].\text{admins}$, **output** (Fetch, Fail, P) to U // Admin P to blame
 - (c) if $\text{isCorrupt}(\text{GID})$ and S is corrupt, **output** resp^* to U // Corrupt member & server can present any group data
- FG.8 if $m = \text{ALLOW}$:
 - (a) set $T_{\text{Ver}}[\text{GID}][\text{UID}] \leftarrow \text{ver}$ // Remember which version the user last fetched
 - (b) for each $(\text{UID}', *)$ in $T_G[\text{GID}][\text{ver}].\text{members}$, add U to $T_{\text{Acq}}[\text{UID}']$ // U can now add members to other groups
 - (c) **output** (Fetch, GID, $T_G[\text{GID}][\text{ver}].\text{attr}$, $T_G[\text{GID}][\text{ver}].\text{members}$, $T_G[\text{GID}][\text{ver}].\text{invited}$, $T_G[\text{GID}][\text{ver}].\text{log}$) to UID

On input (AddMember, GID, UID', admn') from U:

- AM.1 $(\text{UID}, \text{ver}, *, m) \leftarrow \text{prepQuery}(\text{U}, \text{UID}', \text{GID}, \text{AddMember})$
- AM.2 if $m = \text{DENY}$ or $T_G[\text{GID}]$ undefined, **output** (AddMember, GID, Fail) to S
- AM.3 if $m = \text{ALLOW}$ and $\text{U} \in T_{\text{Acq}}[\text{UID}']$ and $\text{UID} \in T_G[\text{GID}][\text{ver}].\text{admins}$:
 - (a) copy $T_G[\text{GID}][\text{ver} + 1] \leftarrow T_G[\text{GID}][\text{ver}]$ and set $T_{\text{Ver}}[\text{GID}] \leftarrow T_{\text{Ver}}[\text{GID}] + 1$
 - (b) append $(\text{UID}', T_U[\text{UID}']. \text{expKey})$ to list $T_G[\text{GID}][\text{ver}].\text{members}$
 - (c) set $T_G[\text{GID}][\text{ver} + 1].\text{log} \leftarrow (\text{UID}', \text{added by}, \text{UID})$
 - (d) if admn' , add UID' to $T_G[\text{GID}][\text{ver}].\text{admins}$
 - (e) flush $T_G[\text{GID}][\text{ver} + 1].\text{leaveQueue}$

On input (RemoveMember, GID, UID') from U:

- RM.1 $(\text{UID}, \text{ver}, *, m) \leftarrow \text{prepQuery}(\text{U}, \text{UID}', \text{GID}, \text{RemoveMember})$
- RM.2 if $m = \text{DENY}$ or $T_G[\text{GID}]$ undefined, **output** (RemoveMember, GID, Fail) to S
- RM.3 if $m = \text{ALLOW}$ and $\text{UID} \in T_G[\text{GID}][\text{ver}].\text{admins}$ and $(\text{UID}', *) \in T_G[\text{GID}][\text{ver}].\text{members}$:
 - copy $T_G[\text{GID}][\text{ver} + 1] \leftarrow T_G[\text{GID}][\text{ver}]$ and set $T_{\text{Ver}}[\text{GID}] \leftarrow T_{\text{Ver}}[\text{GID}] + 1$
 - set $T_G[\text{GID}][\text{ver} + 1].\text{log} \leftarrow (\text{UID}', \text{removed by}, \text{UID})$
 - remove $(\text{UID}', *)$ from $T_G[\text{GID}][\text{ver} + 1].\text{members}$
 - remove UID' from $T_G[\text{GID}][\text{ver} + 1].\text{admins}$ (if in it)
 - flush $T_G[\text{GID}][\text{ver} + 1].\text{leaveQueue}$

On input (SetAdminRights, UID', b_{adm}) from U:

- TA.1 $(\text{UID}, \text{ver}, \perp, m) \leftarrow \text{prepQuery}(\text{U}, \perp, \text{GID}, \text{SetAdminRights})$
- TA.2 if $m = \text{DENY}$ or $T_G[\text{GID}]$ undefined, **output** (SetAdminRights, GID, Fail) to S
- TA.3 if $m = \text{ALLOW}$ and $\text{UID} \in T_G[\text{GID}][\text{ver}].\text{admins}$ and $(\text{UID}', *) \in T_G[\text{GID}][\text{ver}].\text{members}$ and $(b_{\text{adm}} \text{ or } |T_G[\text{GID}][\text{ver}].\text{admins}| > 1)$:
 - (a) copy $T_G[\text{GID}][\text{ver} + 1] \leftarrow T_G[\text{GID}][\text{ver}]$ and set $T_{\text{Ver}}[\text{GID}] \leftarrow T_{\text{Ver}}[\text{GID}] + 1$
 - (b) if b_{adm} **then** add UID' to $T_G[\text{GID}][\text{ver} + 1].\text{admins}$
 - (c) **else** remove UID' from $T_G[\text{GID}][\text{ver} + 1].\text{admins}$
 - (d) set $T_G[\text{GID}][\text{ver} + 1].\text{log} \leftarrow (\text{UID}, \text{toggleAdmin}, \text{UID}', b_{\text{adm}})$
 - (e) flush $T_G[\text{GID}][\text{ver} + 1].\text{leaveQueue}$

Procedure $\text{isPrivate}(\text{GID}, \text{ver})$: // Is there a corrupt user in the group?

- PR.1 Set $b \leftarrow \text{false}$
- PR.2 if $\exists \text{UID} \in T_G[\text{GID}][\text{ver}].\text{members}$ s.t. $T_U[\text{UID}].\text{owner}$ is corrupt **then** $b \leftarrow \text{true}$
- PR.3 **return** b

Procedure $\text{isCorrupt}(\text{GID}, \text{ver})$: // Is there a corrupt admin in the group?

- CO.1 Set $b \leftarrow \text{false}$
- CO.2 if $\exists \text{UID} \in T_G[\text{GID}][\text{ver}].\text{admins}$ s.t. $T_U[\text{UID}].\text{owner}$ is corrupt **then** set $b \leftarrow \text{true}$
- CO.3 **return** b

Procedure $\text{prepQuery}(\text{U}, \text{UID}', \text{GID}, \text{CMD})$: // Common preparation step

- PQ.1 Find UID s.t. $T_U[\text{UID}].\text{owner} = \text{U}$, **else ignore query** // U never registered
- PQ.2 if $\text{UID}' \neq \perp$ **then** set $\text{U}' \leftarrow T_U[\text{UID}']. \text{owner}$ and **ignore query** if undefined // UID' unknown
- PQ.3 Set $\text{ver} \leftarrow T_{\text{Ver}}[\text{GID}]$ // Leakage, depending on Server and group member corruption:
- PQ.4 if $\text{isPrivate}(\text{GID}, \text{ver})$ and S honest **then send** (CMD) to $\mathcal{A}$
- PQ.5 if $\text{isPrivate}(\text{GID}, \text{ver})$ and S corrupt **then** set $i \leftarrow \text{indexOf}(\text{UID}, T_G[\text{GID}][\text{ver}].\text{members})$ and $j \leftarrow \text{indexOf}(\text{UID}', T_G[\text{GID}][\text{ver}].\text{members})$ and **send** (CMD, GID, i, j) to $\mathcal{A}$
- PQ.6 if $\neg \text{isPrivate}(\text{GID}, \text{ver})$, **send** (CMD, GID, UID, UID') to $\mathcal{A}$
- PQ.7 **receive back** m from $\mathcal{A}$
- PQ.8 **return** $(\text{UID}, \text{ver}, \text{U}', m)$

Figure 15: Ideal private group system $\mathcal{F}_{\text{PGS}}$, basic functions.

On input (CMD, GID, UID') from U, where $\text{CMD} \in \{\text{Invite}, \text{RevokeInvitation}\}$:

- ItG.1 (UID, ver, U', m) $\leftarrow$ prepQuery(U, UID', GID, CMD)
- ItG.2 if $m = \text{DENY}$ or $T_G[\text{GID}]$ undefined, **output** (CMD, GID, Fail) to S
- ItG.3 if $m = \text{ALLOW}$ and $\text{UID} \in T_G[\text{GID}][\text{ver}].\text{admins}$:
 - (a) copy $T_G[\text{GID}][\text{ver} + 1] \leftarrow T_G[\text{GID}][\text{ver}]$ and set $T_{\text{Ver}}[\text{GID}] \leftarrow T_{\text{Ver}}[\text{GID}] + 1$
 - (b) if CMD = Invite: append UID' to $T_G[\text{GID}][\text{ver} + 1].\text{invited}$
 - (c) if CMD = RevokeInvitation: remove UID' from $T_G[\text{GID}][\text{ver} + 1].\text{invited}$
 - (d) if CMD = Invite: set $T_G[\text{GID}][\text{ver} + 1].\log \leftarrow (\text{UID}', \text{invited by, UID}')$
 - (e) if CMD = RevokeInvitation: set $T_G[\text{GID}][\text{ver} + 1].\log \leftarrow (\text{UID}', \text{revoked by, UID}')$
 - (f) flush $T_G[\text{GID}][\text{ver} + 1].\text{leaveQueue}$
 - (g) **output** (CMD, GID, UID) to U'

On input (AcceptInvitation, GID) from U:

- CI.1 (UID, ver, $\perp$, m) $\leftarrow$ prepQuery(U, $\perp$, GID, CMD)
- CI.2 if $m = \text{DENY}$ or $T_G[\text{GID}]$ undefined, **output** (AcceptInvitation, GID, Fail) to S
- CI.3 if $m = \text{ALLOW}$ and $\text{UID} \in T_G[\text{GID}][\text{ver}].\text{invited}$:
 - (a) copy $T_G[\text{GID}][\text{ver} + 1] \leftarrow T_G[\text{GID}][\text{ver}]$ and set $T_{\text{Ver}}[\text{GID}] \leftarrow T_{\text{Ver}}[\text{GID}] + 1$
 - (b) add (UID, $T_U[\text{UID}].\text{expKey}$) to $T_G[\text{GID}][\text{ver} + 1].\text{members}$
 - (c) remove UID from $T_G[\text{GID}][\text{ver} + 1].\text{invited}$
 - (d) set $T_G[\text{GID}][\text{ver} + 1].\log \leftarrow (\text{UID}, \text{joined})$

On input (ToggleJoinLink, GID, b_{tgl} , b_{adm}) from U:

- AJ.1 (UID, ver, $\perp$, m) $\leftarrow$ prepQuery(U, $\perp$, GID, ActivateJoinLink)
- AJ.2 if $m = \text{DENY}$ or $T_G[\text{GID}]$ undefined, **output** (ToggleJoinLink, GID, Fail) to S
- AJ.3 if $m = \text{ALLOW}$ and $\text{UID} \in T_G[\text{GID}][\text{ver}].\text{admins}$:
 - (a) copy $T_G[\text{GID}][\text{ver} + 1] \leftarrow T_G[\text{GID}][\text{ver}]$ and set $T_{\text{Ver}}[\text{GID}] \leftarrow T_{\text{Ver}}[\text{GID}] + 1$
 - (b) set $T_G[\text{GID}][\text{ver} + 1].\text{activeLink} \leftarrow b_{\text{tgl}}$
 - (c) set $T_G[\text{GID}][\text{ver} + 1].\text{adminApproval} \leftarrow b_{\text{adm}}$
 - (d) set $T_G[\text{GID}][\text{ver} + 1].\log \leftarrow (\text{UID}, \text{changed link}, b_{\text{tgl}}, b_{\text{adm}})$
 - (e) flush $T_G[\text{GID}][\text{ver} + 1].\text{leaveQueue}$

On input (JoinViaLink, GID) from U:

- JvL.1 (UID, ver, $\perp$, m) $\leftarrow$ prepQuery(U, $\perp$, GID, ActivateJoinLink)
- JvL.2 if $m = \text{DENY}$ or $T_G[\text{GID}]$ undefined, **output** (JoinViaLink, GID, Fail) to S
- JvL.3 if $m = \text{ALLOW}$ and $T_G[\text{GID}][\text{ver}].\text{activeLink}$ and $\neg T_G[\text{GID}][\text{ver}].\text{adminApproval}$:
 - (a) copy $T_G[\text{GID}][\text{ver} + 1] \leftarrow T_G[\text{GID}][\text{ver}]$ and set $T_{\text{Ver}}[\text{GID}] \leftarrow T_{\text{Ver}}[\text{GID}] + 1$
 - (b) append (UID, $T_U[\text{UID}].\text{expKey}$) to list $T_G[\text{GID}][\text{ver} + 1].\text{members}$
 - (c) set $T_G[\text{GID}][\text{ver} + 1].\log \leftarrow (\text{UID}, \text{joined})$
- JvL.4 if $m = \text{ALLOW}$ and $T_G[\text{GID}][\text{ver}].\text{activeLink}$ and $T_G[\text{GID}][\text{ver}].\text{adminApproval}$:
 - (a) record (JoinViaLink, GID, UID)

On input (ApproveJoin, GID, UID') from U:

- AJ.1 (UID, ver, $\perp$, m) $\leftarrow$ prepQuery(U, $\perp$, GID, ActivateJoinLink)
- AJ.2 if $m = \text{DENY}$ or $T_G[\text{GID}]$ undefined, **output** (ApproveJoin, GID, Fail) to S
- AJ.3 if $m = \text{ALLOW}$ and $T_G[\text{GID}][\text{ver}].\text{activeLink}$ and $\exists$ record (JoinViaLink, GID, UID') and $\text{UID} \in T_G[\text{GID}][\text{ver}].\text{admins}$:
 - (a) copy $T_G[\text{GID}][\text{ver} + 1] \leftarrow T_G[\text{GID}][\text{ver}]$ and set $T_{\text{Ver}}[\text{GID}] \leftarrow T_{\text{Ver}}[\text{GID}] + 1$
 - (b) append (UID', $T_U[\text{UID}']. \text{expKey}$) to list $T_G[\text{GID}][\text{ver} + 1].\text{members}$
 - (c) set $T_G[\text{GID}][\text{ver} + 1].\log \leftarrow (\text{UID}', \text{added by, UID})$
 - (d) flush $T_G[\text{GID}][\text{ver} + 1].\text{leaveQueue}$

On input (Refresh, expKey*) from U:

- UP.1 find UID s.t. $T_U[\text{UID}].\text{owner} = \text{U}$, otherwise drop the query
- UP.2 if S corrupt, send (Refresh, UID) to $\mathcal{A}$, else send (Refresh) to $\mathcal{A}$; receive back m
- UP.3 if $m = \text{DENY}$ **output** (Refresh, UID, Fail) to S
- UP.4 if $m = \text{ALLOW}$ do:
 - (a) flush $T_{\text{Acq}}[\text{UID}]$
 - (b) $\text{expKey}_{\text{new}} \xleftarrow{\$} \{0, 1\}^\lambda$
 - (c) if U is corrupt, set $\text{expKey}_{\text{new}} \leftarrow \text{expKey}^*$
 - (d) for all GID s.t. $(\text{UID}, \text{expKey}_{\text{old}}) \in T_G[\text{GID}][T_{\text{Ver}}[\text{GID}]].\text{members}$, remove $(\text{UID}, \text{expKey}_{\text{old}})$ from that list and add $(\text{UID}, \text{expKey}_{\text{new}})$ instead
 - (e) if $\exists \text{GID}$ s.t. $(\text{UID}, *) \in T_G[\text{GID}][T_{\text{Ver}}[\text{GID}]].\text{members}$ and $\neg \text{isPrivate}(\text{GID})$, send $\text{expKey}_{\text{new}}$ to $\mathcal{A}$.
 - (f) set $T_U[\text{UID}].\text{expKey} \leftarrow \text{expKey}_{\text{new}}$
 - (g) if U honest, **output** $\text{expKey}_{\text{new}}$ to U

On input (Leave, GID) from U:

- LG.1 (UID, ver, $\perp$, m) $\leftarrow$ prepQuery(U, $\perp$, GID, Leave)
- LG.2 if $m = \text{DENY}$ or $T_G[\text{GID}]$ undefined **output** (Leave, GID, Fail) to S
- LG.3 if $m = \text{ALLOW}$ and $(\text{UID}, *) \in T_G[\text{GID}][\text{ver}].\text{members}$:
 - (a) copy $T_G[\text{GID}][\text{ver} + 1] \leftarrow T_G[\text{GID}][\text{ver}]$ and set $T_{\text{Ver}}[\text{GID}] \leftarrow T_{\text{Ver}}[\text{GID}] + 1$
 - (b) append UID to $T_G[\text{GID}][\text{ver} + 1].\text{leaveQueue}$
 - (c) remove $(\text{UID}, *)$ from $T_G[\text{GID}][\text{ver} + 1].\text{members}$
 - (d) set $T_G[\text{GID}][\text{ver} + 1].\log \leftarrow (\text{UID}, \text{left})$

On input (UserAdd, GID, UID') from U:

- UA.1 (UID, ver, *, m) $\leftarrow$ prepQuery(U, UID', GID, UserAdd)
- UA.2 if $m = \text{DENY}$ or $T_G[\text{GID}]$ undefined, **output** (UserAdd, GID, Fail) to S
- UA.3 if $m = \text{ALLOW}$ and $\text{U} \in T_{\text{Acq}}[\text{UID}']$ and $T_G[\text{GID}][\text{ver}].\text{allowUserAdd}$:
 - (a) copy $T_G[\text{GID}][\text{ver} + 1] \leftarrow T_G[\text{GID}][\text{ver}]$ and set $T_{\text{Ver}}[\text{GID}] \leftarrow T_{\text{Ver}}[\text{GID}] + 1$
 - (b) append $(\text{UID}', T_U[\text{UID}']. \text{expKey})$ to list $T_G[\text{GID}][\text{ver} + 1].\text{members}$
 - (c) set $T_G[\text{GID}][\text{ver} + 1].\log \leftarrow (\text{UID}', \text{added by, UID})$

On input (ChangeAttr, GID, attr) from U:

- GA.1 (UID, ver, $\perp$, m) $\leftarrow$ prepQuery(U, $\perp$, GID, ChangeAttr)
- GA.2 if $m = \text{DENY}$ or $T_G[\text{GID}]$ undefined **output** (ChangeAttr, GID, Fail) to S
- GA.3 if $m = \text{ALLOW}$ and $((\text{UID}, *) \in T_G[\text{GID}][\text{ver}].\text{members} \wedge T_G[\text{GID}][\text{ver}].\text{allowUserAttr})$ or $(\text{UID}, *) \in T_G[\text{GID}][\text{ver}].\text{admins}$:
 - copy $T_G[\text{GID}][\text{ver} + 1] \leftarrow T_G[\text{GID}][\text{ver}]$ and set $T_{\text{Ver}}[\text{GID}] \leftarrow T_{\text{Ver}}[\text{GID}] + 1$
 - set $T_G[\text{GID}][\text{ver} + 1].\text{attr} \leftarrow \text{attr}$
 - set $T_G[\text{GID}][\text{ver} + 1].\log \leftarrow (\text{UID}, \text{changed attr})$
 - if $(\text{UID}, *) \in T_G[\text{GID}][\text{ver}].\text{admins}$ then flush $T_G[\text{GID}][\text{ver} + 1].\text{leaveQueue}$

On input (ChangeSettings, GID, b_{attr} , b_{add}) from U:

- GS.1 (UID, ver, $\perp$, m) $\leftarrow$ prepQuery(U, $\perp$, GID, ChangeSettings)
- GS.2 if $m = \text{DENY}$ or $T_G[\text{GID}]$ undefined, **output** (ChangeSettings, GID, Fail) to S
- GS.3 if $m = \text{ALLOW}$ and $\text{UID} \in T_G[\text{GID}][\text{ver}].\text{admins}$:
 - (a) copy $T_G[\text{GID}][\text{ver} + 1] \leftarrow T_G[\text{GID}][\text{ver}]$ and set $T_{\text{Ver}}[\text{GID}] \leftarrow T_{\text{Ver}}[\text{GID}] + 1$
 - (b) set $T_G[\text{GID}][\text{ver} + 1].\text{allowUserAttr} \leftarrow b_{\text{attr}}$
 - (c) set $T_G[\text{GID}][\text{ver} + 1].\text{allowUserAdd} \leftarrow b_{\text{add}}$
 - (d) set $T_G[\text{GID}][\text{ver} + 1].\log \leftarrow (\text{UID}, \text{changed conf}, b_{\text{attr}}, b_{\text{add}})$
 - (e) flush $T_G[\text{GID}][\text{ver} + 1].\text{leaveQueue}$

Figure 16: Private group system $\mathcal{F}_{\text{PGS}}$, extended functions.

Privacy properties	Satisfied?	Why?
Group membership is hidden from the server	✓{S}	encrypted under group key
Group information (e.g., image and name) is hidden from the server.	✓{S}	encrypted under group key
After a user is removed by an admin, they are no longer able to decrypt the group list	✓{U}	key rotation (RotateGroupKey) (but mal. Server can ignore rotate updated)
A user leaving the group is no longer able to decrypt the group list...		
– immediately	✓{U}	Fetch via σ^{psdn} required after leave. Mal. server could still deliver group list and key is not rotated yet.
– after the first group member went online	✓{U}	as above
– after the first admin went online	✓{U}	Honest admin would rotate key. But malicious server/admin can ignore rotate update
Group sizes are hidden from the server	✗	List length leaks
A user can revoke the access of some users to their (future) profile information.	✓{}	Can do uk rotation (Refresh). Relies on server accepting and acquaintances not leaking new key. The users that have their access revoked may be malicious.
Profile information of a user A is hidden from ...		
– the server	✓{S}	encrypted under expKey
– users who share a group with user A	✗	In a group, you can see the profile information of all other members.
– users who once shared a group with user A	✗/✓{U,A}	Until rotation of uk, the old group members still have uk. After rotate, encrypted under new expKey . Server chooses UID
User registration cannot be attributed to a UID	✗	
User privacy: Fetching of a group		
– is unlinkable across groups	✓{U,A,S}/✓{S}	Until a leave, only one fet per group (not per user). After a leave vk^{psdn} allows group members to link to UID.
– is unlinkable within the same group	✓{U,A,S}/✗	Until a leave, only one fet per group (not per user). After a leave vk^{psdn} allows to link fetching to an index.
– cannot be attributed to a UID	✓{U,A,S}/✓{S}	Until a leave, only one fet per group (not per user). After a leave vk^{psdn} allows group members to link to UID.
User privacy: Being removed, added or invited by an admin		
– is unlinkable across groups	✓{S}	encrypted UID
– is unlinkable within the same group	✗/✓{S}	Readdition unlinkable because commitments are randomized and encryption involves nonces. Adding and removing (the same line) is linkable.
– cannot be attributed to a UID	✓{S}	encrypted UID
User privacy: Leaving a group and joining via invite link		
– is unlinkable across groups	✓{S}	UID encrypted and vk^{psdn} per group.
– is unlinkable within the same group	✗	re-joining is linkable because same rrs leads to same vk^{psdn} . Joining-then-leaving is linkable (same line).
– cannot be attributed to a UID	✓{S}	uid encrypted and vk^{psdn} per group
Admin privacy: actions (Create, AddMember, RemoveMember, DeleteGroup) of an admin are		
– unlinkable across groups	✓{S}	different vk^{psdn} per group
– unlinkable within the same group	✗	line index leaks this
– cannot be attributed to a UID	✓{S}	uid encrypted and vk^{psdn} per group
Admin privacy: creating invitation links		
– is unlinkable across groups	✓{S}	link does not contain admin info
– cannot be attributed to a UID	✓{S}	link does not contain admin info
Consistency properties	Satisfied?	Why?
Only admins can add, invite, or remove users	✓{U,S}	σ^{psdn} required for changes to main list. Fetch requires to check admin flag (even against old version). σ^{psdn} allows to trace changes of admn .
Only admins can grant admin privileges	✓{U,S}	Fetch requires to check admin flag (even against old version). σ^{psdn} allows to trace changes of admn .
Only acquaintances of a user can be added to a group by that user	✓{U,A}	Server provides signature on acq in GrantAcquaintance . Mal. server can deny response.

Figure 17: Private group system properties (Part I).

Sending and Spam Protection (Endorsements)	Satisfied?	Why?
The server does not know which user sent a group message.	✓ $\{U,A,S\}$	Sending tokens
The server cannot link a group message to a group.	✓ $\{U,A,S\}$	Sending token independent of group
The server cannot link two distinct messages for the same group	✓ $\{U,A,S\}$	Sending token independent of group
A server can recognize an illegitimate group message (i.e., from a sender to a receiver who are not acquainted)	✓	Sending requires knowing all recipient's snd .
Blaming	Satisfied?	Why?
If the group description is changed by the server and that change was illegitimate, then the users know that it's the server's fault.	✓ $\{S\}$	Server cannot forge σ^{psdn}
Given an semi-honest server, the users know which admin made a flawed update	✓ $\{A\}$	Server checks σ^{psdn} . In particular, admin index in the signature
Even a malicious admin and a malicious server colluding cannot frame another admin for producing a valid group state	✓ $\{A,S\}$	Cannot forge σ^{psdn} under that admins vk^{psdn} .
Rollback attacks	Satisfied?	Why?
Users/clients cannot be rolled back to an older group version than the last one they have seen.	✓ $\{S\}$	version number signed by σ^{psdn}
The server cannot present different valid group configurations to different users.	✗	Server can withhold updates form certain users.
DoS Properties	Satisfied?	Why?
The server can identify a legitimate request to fetch a group description.	✓ $\{\}$	Fetching only with fetching token or σ^{psdn} .
Non-admin group members cannot add invalid entries to the group member list.	✓ $\{S\}$	need vk^{psdn} for a line with admn set to change. (Does not hold for the appendix)
Admins cannot make users store invalid entries of the group member list	✓ $\{A\}$	Users can identify invalid entries (checking σ^{psdn} , C_{UID} , and using GrantAcquaintance) and ignore them.
Post-quantumness	Satisfied?	Why?
Group member lists are protected from harvest-now-decrypt-later attacks.	✓ $\{S\}$	Symmetric group encryption + stat. hiding commitments.
Performing group actions is protected against quantum attackers.	✓/✗	depending on the instantiation of the (key rerandomizable) signatures.

Figure 18: Private group system properties (Part II).

b_{attr} , b_{add} , and attr . The simulator can do this by observing the ideal cipher. For a given ciphertext c , the simulator can look for a corresponding Enc query leading to c . Collisions happen only with negligible probability. In fact, the ideal cipher outputs have to be authenticated to allow the simulator distinguishing properly encrypting ciphertexts from erroneous ones to check that the obtained ciphertext does encrypt to attributes and settings. The simulator gives the extracted values to $\mathcal{F}_{\text{PGS}}$ via the Create interface on behalf of the corrupt user. Finally, the simulator checks σ^{psdn} using the provided vk^{psdn} . If the signature is valid, the simulator gives ALLOW to the functionality, else DENY . Also, the simulator stores the encrypted group list like a real server, if the signature verifies.

Corrupt server, honest user: The simulator uses that the encryption scheme is modeled as an ideal cipher (IC), and the commitment is equivocable. That is, it can produce a group header and a member list without knowing attr and UID . If the group gets corrupted later, the simulator learns the attributes and the user secrets stored in this group and can program the IC, and equivocate the commitment accordingly. The fetching token can be a random string. To simulate the pseudonym signature, the simulator retrieves $(\text{sk}_{\text{base}}, \text{vk}_{\text{base}})$ that it created for this user when simulating registration. Next, the simulator picks a group key gk and derives vk^{psdn} and σ^{psdn} like in the protocol description using sk_{base} and gk .

Simulation of AddMember *Honest server, corrupt user:* Like in Create , the simulator can extract UID' from the new member ciphertext mct , using the IC. The GID and the new admin flag admn' are provided in plain. This allows the simulator to give input $(\text{AddMember}, \text{GID}, \text{UID}', \text{admn}')$ to $\mathcal{F}_{\text{PGS}}$ on behalf of the corrupt user. Extracting UID' from the IC also yields the ek that is used by the corrupt user. The simulator, thus, can check if there was a $\text{KDF}(\text{gk}, \text{"SUB-KEY"})$ query that evaluated to (ek, rrs) . If not such query exists, the other group members reject the update with overwhelming probability because either they derive a different rrs and the pseudonym signature does not verify or they derive a different ek and decryption of mct fails. Therefore, the simulator can ignore this query. If such a record exists, the simulator first uses ek to decrypt mct_i , where i is the index of the changing admin, i.e., the value specified in core-sig . With that, the simulator can look up the vk_{base} that was stored at registration for UID and derive $\text{vk}^{\text{psdn}} \leftarrow \text{RandVK}(\text{vk}_{\text{base}}, \text{rrs})$. Again, like in Create , the simulator verifies σ^{psdn} and if verification succeeds, he returns ALLOW to $\mathcal{F}_{\text{PGS}}$ and else DENY to ensure that the server outputs a fail.

Corrupt server, honest user: Like in Create , the simulator uses the equivocable commitment scheme and the IC

to produce a new entry without knowing UID' . Then, the simulator retrieves the gk that it simulated at creation to derive vk^{psdn} and σ^{psdn} like in the real protocol.

Simulation of SetAdminRights, RemoveMember These protocols works similar to AddMember . We only describe the differences in the simulation:

- In RemoveMember , the user cannot decrypt a new entry to extract UID' but it has to compare the new member list with the old one and decrypt the one mct that is missing in the new version.
- After RemoveMember , the simulator must also simulate a RotateGroupKey , which works exactly like simulating Create .
- To simulate SetAdminRights , the simulator can simply read the changed admin flag from the new group list. It extracts UID' like in AddMember .

Simulation of GrantAcquaintance If the user U' who receive the uk from U is corrupted, but U is not, the simulator must now program $\text{KDF}(\text{uk}, \text{"ACQ-Tag"}) \leftarrow \text{acq}$ and $\text{KDF}(\text{uk}, \text{"EXP-Key"}) \leftarrow \text{expKey}$, as uk is leaked to the environment. For simulating a honest server, note that the server holds no particular secrets and thus, the simulator can just run the real server's code. Finally, to simulate an honest user receiving $(\text{vk}_{\text{base}}, \text{acq}, \sigma)$, the simulator can also receive $\text{vk}_{\text{UID}}^{\text{pers}}$ from (the internally simulated $\mathcal{F}_{\text{PKI}}$) and verify the signature. Depending on the result, the simulator either gives DENY or ALLOW to $\mathcal{F}_{\text{PGS}}$.

Simulation of Fetch *Honest server, corrupt user:* The simulator also checks internally, if it stored a record corresponding to the required GID . If that is the case, the entry corresponds to a corrupt group, i.e., a group where at least one member is corrupt. The simulator just returns the created group list like a real server also does. As this group is either created by the adversary or was already equivocated when handing gk to $\mathcal{A}$, the simulator can just behave like an honest server. If instead the corrupt user is requesting a GID is an honest group, the adversary does not know fet and also cannot forge a σ^{psdn} , by the security of the key rerandomizable signature scheme. If no corresponding fet is stored for GID , the simulator can give input DENY to $\mathcal{F}_{\text{PGS}}$ to trigger a fail output of the server.

Corrupt server, honest user: The simulator must perform all consistency checks detailed in UsrVfyUpdate . First, the simulator can again use the IC to extract a group key gk that was used to create the received group list. Using this, the simulator is able to compute $\text{vk}_i^{\text{psdn}}$, where i is the admin specified in core-sig . If σ^{psdn} does not verify under this key, the server is to blame, i.e., the

simulator can give input (ATTACK, S, r^*) to $\mathcal{F}_{\text{PGS}}$, where r^* is the view of the group that the simulator decrypted. If the signature verifies, but any of the other checks fail, the simulator can give $(\text{ATTACK}, \text{GetUID}(i, \text{mem-1st}))$ to $\mathcal{F}_{\text{PGS}}$, indicating that the i -th admin is to blame for the failure. By the unforgeability of the key rerandomizable signature scheme, the adversary can not frame any other admin than the one extracted by the simulator.

Simulation of adaptive user corruption When a user U gets corrupted, the simulator must equivocate the commitments and all mcts for all groups where U is a member. The functionality gives the simulator all the required UIDs and the simulator can also use the uk that it picked for this honest user upon registration.